

Title Page

Lab 2 — HTTP Analysis Using Wireshark: Embedded Image Traffic

Student Name: Athanasius Orikeze Alekwe

Student ID: 2025/FWSD/11230

**Course Name: Basic Networking Skills for Digital
Forensics**

Instructor Name: Aminu Idris

Date of Submission: 10/09/2026

INTRODUCTION

HTTP is one of the main protocols used to transfer web content between clients and web servers. A webpage may appear to the user as a single page, but the browser can make several separate requests in the background for the different resources required to display it. In this practical exercise, I examined this behaviour by hosting a simple plaintext HTTP webpage containing an embedded JPEG image on a local Apache web server.

The practical was carried out in an isolated Kali Linux virtual machine. The webpage, `image.html`, contained a reference to `lab_photo.jpg`. I captured the browser-generated traffic on the loopback interface and examined the separate HTTP requests produced when Firefox loaded the page. The captured image response was then analysed at TCP level to determine how the object was divided across segments and later reassembled.

The image was recovered directly from the packet capture using TShark and Wireshark's HTTP object-export function. Finally, I calculated SHA-256 hashes for the original and recovered images to determine whether the recovered object was identical to the source. A separate curl capture was also performed to demonstrate the difference between a normal web browser and a command-line HTTP client.

This procedure follows the purpose of the laboratory exercise, which requires the student to prove that a browser can request multiple objects, analyse the TCP segments carrying an image response, extract the image, and verify its integrity.

Objectives

The objectives of this practical were to:

Host a locally generated plaintext HTTP webpage containing an authorised JPEG image.

Capture the HTTP communication created when the page was opened in a web browser.

Demonstrate that loading one webpage can result in multiple HTTP GET requests.

Identify the requests and responses associated with image.html and lab_photo.jpg.

Examine the TCP segments involved in transferring the JPEG image.

Compare IP packet size, TCP header size, TCP payload size, HTTP headers and HTTP object size.

Demonstrate TCP reassembly of the JPEG response.

Extract the transferred image from the PCAPNG using both TShark and Wireshark.

Calculate SHA-256 hashes for the source and recovered image and compare them.

Explain the effect of browser caching and why a private browsing session was used.

Compare Firefox behaviour with curl and explain why curl did not automatically request the embedded JPEG.

Laboratory / Tools Used

The practical was completed using Kali Linux 2025.3 running inside VMware Workstation. Apache2 was used to provide the local HTTP service, while Firefox was used to generate the browser traffic. A private Firefox window was used during the assessed capture to reduce the possibility of the embedded image being loaded from browser cache instead of being requested from the server.

Wireshark and TShark were used for packet analysis, HTTP filtering, TCP segmentation analysis, TCP conversation statistics, and HTTP object extraction. curl was used for HTTP header verification and for the browser-versus-command-line comparison. ImageMagick identify and the Linux file command were used to inspect the JPEG format and dimensions. sha256sum was used for evidence-integrity calculations, while cmp and stat were used during the final recovered-image verification.

The web server operated locally at 127.0.0.1 on TCP port 80, and the packet capture was taken from the loopback interface lo. The official manual identifies Apache2, Wireshark, TShark, Firefox/Chrome/Edge, curl, file, identify, and sha256sum as the relevant tools for the exercise.

EXECUTIVE SUMMARY AND SCOPE

This practical investigated the transfer and recovery of an embedded JPEG image delivered over plaintext HTTP. A simple HTML page called image.html was hosted by Apache on 127.0.0.1. The page referenced lab_photo.jpg, a JPEG image measuring 1374 × 1145 pixels.

The browser capture demonstrated that Firefox did not treat the page as a single network object. It requested /image.html first, followed by /lab_photo.jpg after parsing the HTML. Firefox also generated a separate /favicon.ico request, which returned 404 Not Found because no favicon had been configured.

The image response was reconstructed from five TCP segments with a combined TCP application payload of 184,535 bytes. The JPEG itself had a final verified HTTP Content-Length of 184,245 bytes, leaving 290 bytes attributable to the HTTP response header block. These exact segmentation values are recorded in the uploaded report.

The JPEG was successfully exported from the capture and opened normally. Its SHA-256 hash was identical to the source-image SHA-256, demonstrating byte-for-byte recovery. A second capture using curl showed only GET /image.html, confirming that curl did not automatically fetch the image referenced inside the HTML.

The scope was limited entirely to an authorised local environment. No third-party systems, public websites, credentials, personal communications or production-network traffic were captured.

EVIDENCE ACQUISITION AND INTEGRITY HASHES

Before generating traffic, I created separate directories for original evidence, working copies, exported objects, reports, screenshots, and scripts. This helped keep the original evidence separated from files created during analysis, as required by the laboratory procedure.

The HTML page and source image were placed in Apache's /var/www/html/ directory. Baseline SHA-256 hashes were calculated before the main capture. The original image SHA-256 was:

d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

The browser-generated traffic was captured to:

evidence/image_traffic.pcapng

A working copy was then created as:

working/image_traffic_working.pcapng

The final browser evidence PCAPNG and its working copy produced the same SHA-256:

f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

The identical values confirmed that the working copy was a byte-for-byte copy of the preserved evidence at the point it was created.

The curl-only capture was stored separately as:

evidence/curl_only.pcapng

Its recorded SHA-256 was:

0871419547cf657d615a42e9f80676c416d2a9d55e4dfd767963f04b63fe59cd

Analysis was carried out on the working browser capture rather than modifying the original evidence. Earlier trial captures were moved into archive_attempts, leaving the final evidence folders clean while still preserving the previous attempts. This is consistent with the manual's instruction to preserve original capture files and perform analysis on verified working copies.

HTTP REQUEST/RESPONSE INVENTORY

The final browser capture contained three HTTP requests on TCP stream 0.

The first relevant request was recorded in frame 4 at:

20:07:00.766361217 WAT

Firefox requested:

GET /image.html

Apache responded successfully with HTTP 200 OK.

After Firefox received and parsed the HTML page, it found the embedded reference and generated another request. The image request appeared in frame 8 at:

20:07:00.932501697 WAT

The requested resource was:

GET /lab_photo.jpg

The reconstructed HTTP image response was identified at frame 17 and returned:

HTTP/1.1 200 OK

Content-Type: image/jpeg

Content-Length: 184245

Firefox also generated:

GET /favicon.ico

in frame 19. Apache responded in frame 20 with 404 Not Found, since no favicon had been configured.

The favicon does not represent an error in the practical. It is an example of an extra browser-generated resource request, and the official manual specifically states that a favicon or similar request may appear and should be documented rather than removed.

These results demonstrate why visiting what appears to be one webpage can produce multiple HTTP transactions: the initial HTML document can contain references to other resources that the browser subsequently retrieves.

TCP SEGMENTATION AND REASSEMBLY ANALYSIS

Frame	IP Length	IP Header	TCP Header	TCP Payload
9	32,820	20	32	32,768
11	32,820	20	32	32,768
13	47,220	20	32	47,168
14	47,220	20	32	47,168
17	24,715	20	32	24,663

Adding the TCP payload values gives:

$32,768 + 32,768 + 47,168 + 47,168 + 24,663 = 184,535$ bytes

TShark independently reported:

Reassembled TCP length: 184,535 bytes

Segment count: 5

These figures are recorded directly in the practical evidence.

The relationship between the protocol sizes can also be demonstrated using individual packets. For frame 9:

20-byte IP header + 32-byte TCP header + 32,768-byte TCP payload = 32,820-byte IP length

For frame 13:

$20 + 32 + 47,168 = 47,220$ bytes

For frame 17:

$20 + 32 + 24,663 = 24,715$ bytes

The combined TCP application payload was 184,535 bytes, whereas the JPEG Content-Length was 184,245 bytes. Therefore:

$184,535 - 184,245 = 290$ bytes

The additional 290 bytes represent the HTTP response header block accompanying the JPEG body. In other words, tcp.len accounts for HTTP application data carried inside TCP, while HTTP Content-Length describes the entity body itself and does not include the HTTP response headers.

The unusually large observed TCP payload values are also significant. The capture was performed on the local loopback interface, where operating-system buffering and segmentation/offloading behaviour can result in captured segments that are much larger than the packet sizes usually associated with a physical Ethernet MTU. The official manual explicitly warns that capture location, MTU, operating-system buffering and offloading can affect observed segment sizes.

CURL VERSUS BROWSER EXPLANATION

The browser and curl behaved differently because they perform different functions.

When Firefox loaded /image.html, it did more than download the HTML document. It parsed the page and discovered that the HTML referenced lab_photo.jpg. Firefox therefore sent another HTTP request for the JPEG. It also attempted to retrieve /favicon.ico.

The curl test behaved differently. The command was:

```
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
```

The separate curl capture showed only:

```
GET /image.html
```

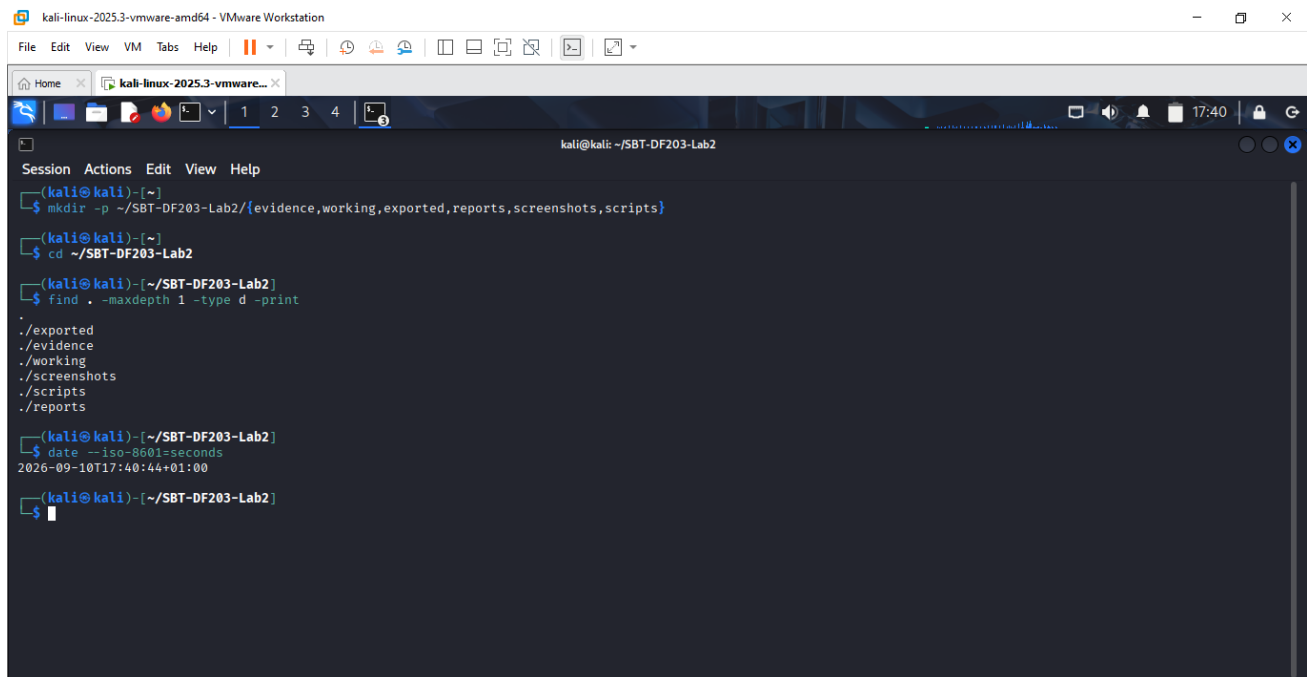
No request for /lab_photo.jpg appeared.

This happened because ordinary curl retrieves the resource explicitly provided on the command line. It receives the reference as part of the HTML content, but it does not behave as a browser rendering engine and automatically request every embedded object.

The comparison therefore provides direct evidence that the multiple requests observed in the Firefox capture resulted from browser processing of the HTML document rather than from a requirement that HTTP itself retrieve all resources automatically. This is the behaviour the manual expects the curl comparison to demonstrate.

Practical Evidence

Create the forensic lab folders



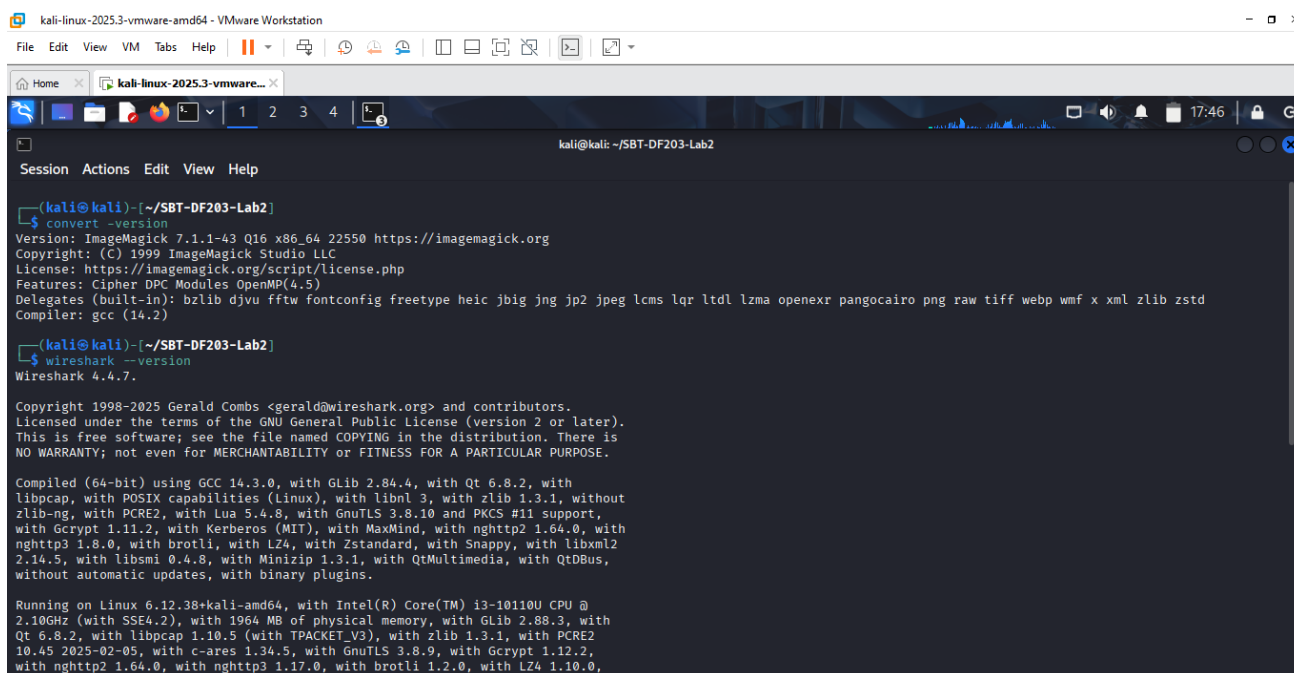
```
kali@kali: ~/$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
kali@kali: ~/$ cd ~/SBT-DF203-Lab2
kali@kali: ~/SBT-DF203-Lab2/$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
kali@kali: ~/SBT-DF203-Lab2/$ date --iso-8601=seconds
2026-09-10T17:40:44+01:00
kali@kali: ~/SBT-DF203-Lab2/$
```

Fig 1.0 Lab folder structure and start time.

Evidence field	Entry
Case/Lab Identifier	SBT-DF203-Lab2-Athanasius-Alekwe
Trainee Name	Athanasius Alekwe
Registration Number	2025/FWSD/11230
Time Started:	2026-09-10T17:40:44+01:00

Explanation

This screenshot shows the creation of the laboratory folder structure used to separate original evidence, working files, exported objects, reports, screenshots and scripts. The displayed time also establishes when I began the practical and provides an initial record for the evidence worksheet.



```
kali@kali: ~/SBT-DF203-Lab2
$ convert --version
Version: ImageMagick 7.1.1-43 Q16 x86_64 22550 https://imagemagick.org
Copyright: (C) 1999 ImageMagick Studio LLC
License: https://imagemagick.org/script/license.php
Features: Cipher DPC Modules OpenMP(4.5)
Delegates (built-in): bzlib djvu fftw fontconfig freetype heic jbig jng jp2 jpeg lcms lqr ltdl lzma openexr pangocairo png raw tiff webp wmf x xml zlib zstd
Compiler: gcc (14.2)

$ wireshark --version
Wireshark 4.4.7.

Copyright 1998-2025 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

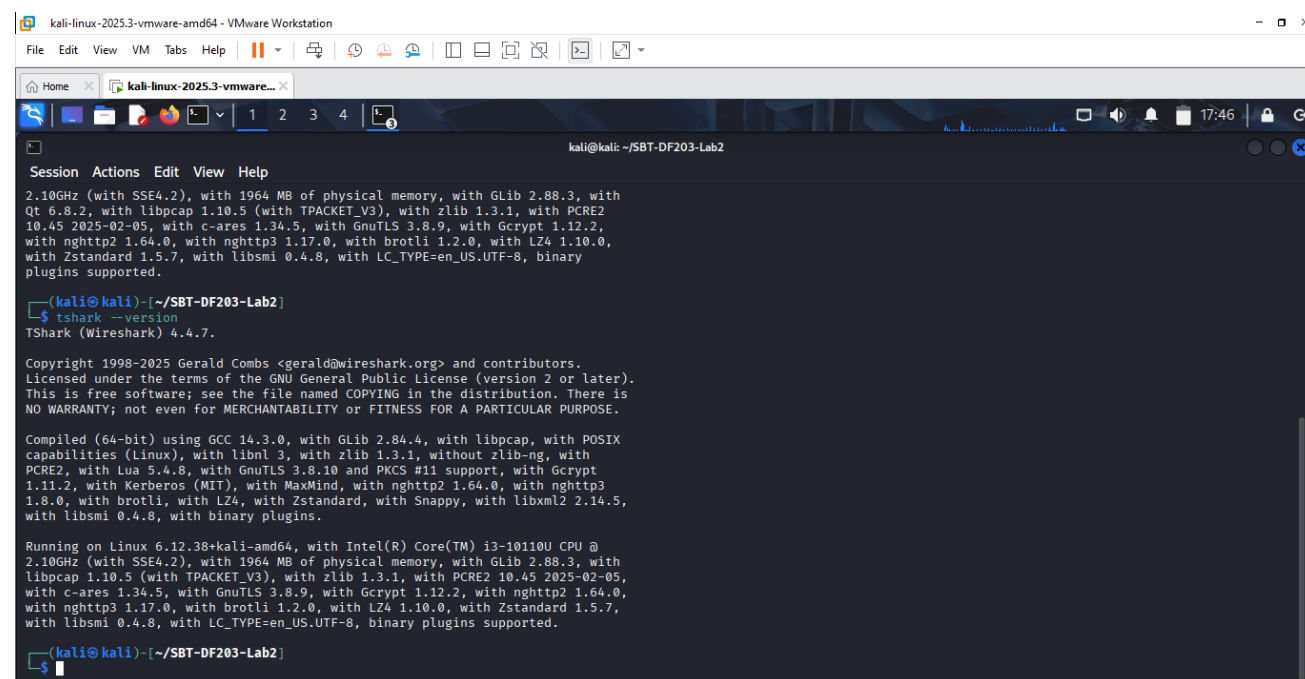
Compiled (64-bit) using GCC 14.3.0, with GLib 2.84.4, with Qt 6.8.2, with
libpcap, with POSIX capabilities (Linux), with libnl 3, with zlib 1.3.1, without
zlib-ng, with PCRE2, with Lua 5.4.8, with GnuTLS 3.8.10 and PKCS #11 support,
with Gcrypt 1.11.2, with Kerberos (MIT), with MaxMind, with nghttp2 1.64.0, with
nghttp3 1.8.0, with brotli, with LZ4, with Zstandard, with Snappy, with libxml2
2.14.5, with libsmi 0.4.8, with Minizip 1.3.1, with QtMultimedia, with QtDBus,
without automatic updates, with binary plugins.

Running on Linux 6.12.38+kali-amd64, with Intel(R) Core(TM) i3-10110U CPU @
2.10GHz (with SSE4.2), with 1964 MB of physical memory, with GLib 2.88.3, with
Qt 6.8.2, with libpcap 1.10.5 (with TPACKET_V3), with zlib 1.3.1, with PCRE2
10.45 2025-02-05, with c-ares 1.34.5, with GnuTLS 3.8.9, with Gcrypt 1.12.2,
with nghttp2 1.64.0, with nghttp3 1.17.0, with brotli 1.2.0, with LZ4 1.10.0,
```

Fig 1.1 Verifying imagemagick and wireshark are installed.

Explanation

This figure confirms that ImageMagick and Wireshark were available on the Kali system before the practical began. Verifying the tools in advance reduced the chance of interrupting the evidence-acquisition process later.



```
kali@kali: ~/SBT-DF203-Lab2
$ tshark --version
TShark (Wireshark) 4.4.7.

Copyright 1998-2025 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

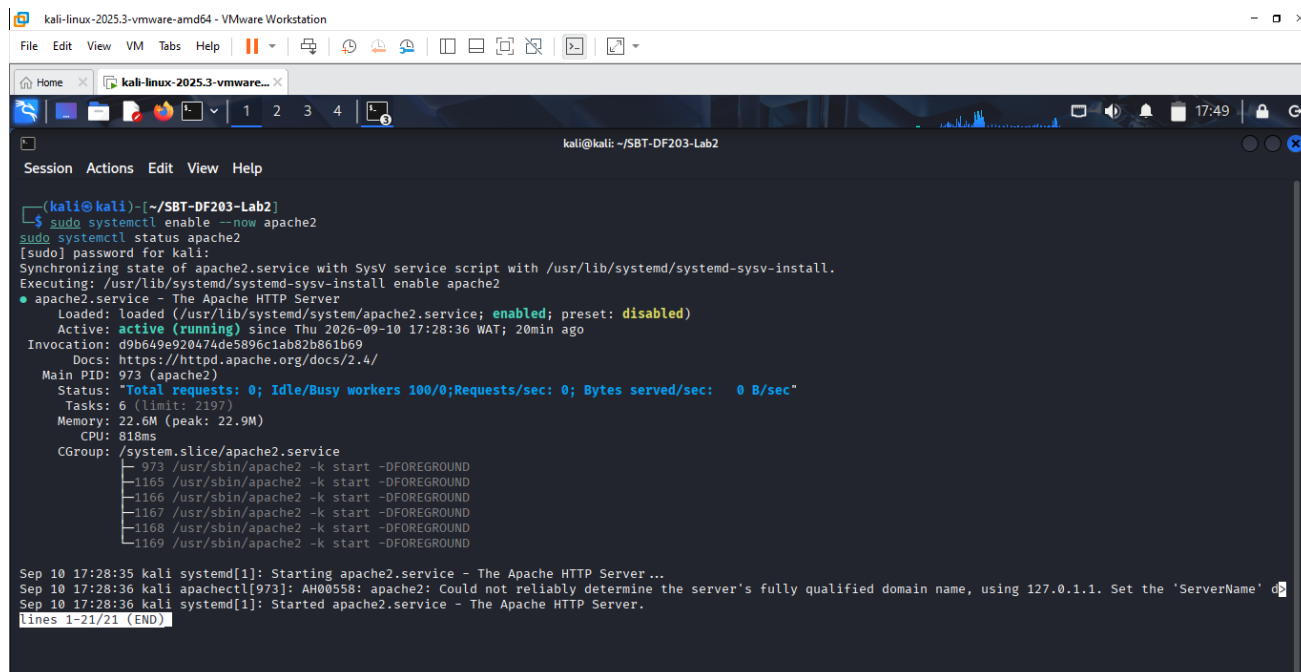
Compiled (64-bit) using GCC 14.3.0, with GLib 2.84.4, with libpcap, with POSIX
capabilities (Linux), with libnl 3, with zlib 1.3.1, without zlib-ng, with
PCRE2, with Lua 5.4.8, with GnuTLS 3.8.10 and PKCS #11 support, with Gcrypt
1.11.2, with Kerberos (MIT), with MaxMind, with nghttp2 1.64.0, with nghttp3
1.8.0, with brotli, with LZ4, with Zstandard, with Snappy, with libxml2 2.14.5,
with libsmi 0.4.8, with binary plugins.

Running on Linux 6.12.38+kali-amd64, with Intel(R) Core(TM) i3-10110U CPU @
2.10GHz (with SSE4.2), with 1964 MB of physical memory, with GLib 2.88.3, with
libpcap 1.10.5 (with TPACKET_V3), with zlib 1.3.1, with PCRE2 10.45 2025-02-05,
with c-ares 1.34.5, with GnuTLS 3.8.9, with Gcrypt 1.12.2, with nghttp2 1.64.0,
with nghttp3 1.17.0, with brotli 1.2.0, with LZ4 1.10.0, with Zstandard 1.5.7,
with libsmi 0.4.8, with LC_TYPE=en_US.UTF-8, binary plugins supported.
```

Fig 1.2 Verifying tshark is installed.

Explanation

The screenshot confirms that TShark was installed and accessible from the terminal. TShark was required for packet capture, HTTP filtering, TCP analysis and automated object extraction during the practical.



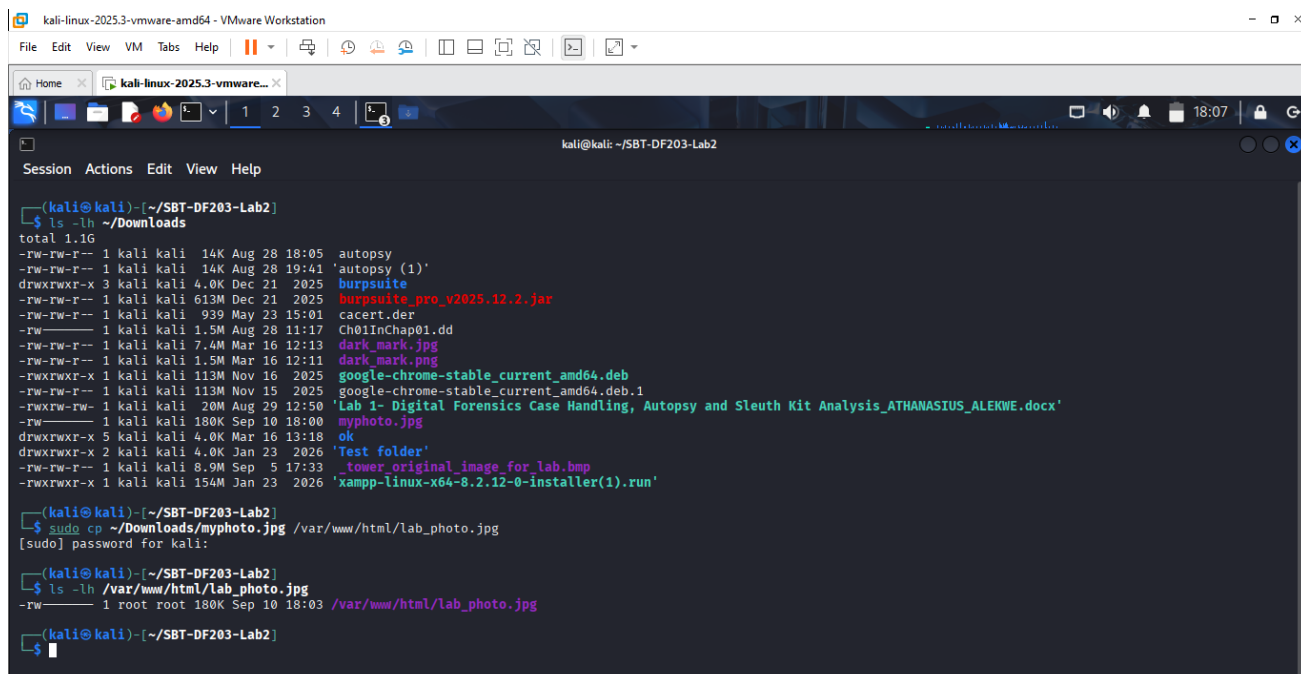
```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$ sudo systemctl enable --now apache2
sudo systemctl status apache2
[sudo] password for kali:
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 17:28:36 WAT; 20min ago
 Invocation: d9b649e920474de5896c1ab82b861b69
    Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 973 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 6 (limit: 2197)
  Memory: 22.6M (peak: 22.9M)
     CPU: 818ms
   CGroup: /system.slice/apache2.service
           └─ 973 /usr/sbin/apache2 -k start -DFOREGROUND
             1165 /usr/sbin/apache2 -k start -DFOREGROUND
             1166 /usr/sbin/apache2 -k start -DFOREGROUND
             1167 /usr/sbin/apache2 -k start -DFOREGROUND
             1168 /usr/sbin/apache2 -k start -DFOREGROUND
             1169 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 10 17:28:35 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 10 17:28:36 kali apachectl[973]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName'
Sep 10 17:28:36 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

Fig 1.3 Starting Apache page.

Explanation

This figure shows Apache being enabled and started successfully. The active service provided the local HTTP server used to host image.html and lab_photo.jpg.



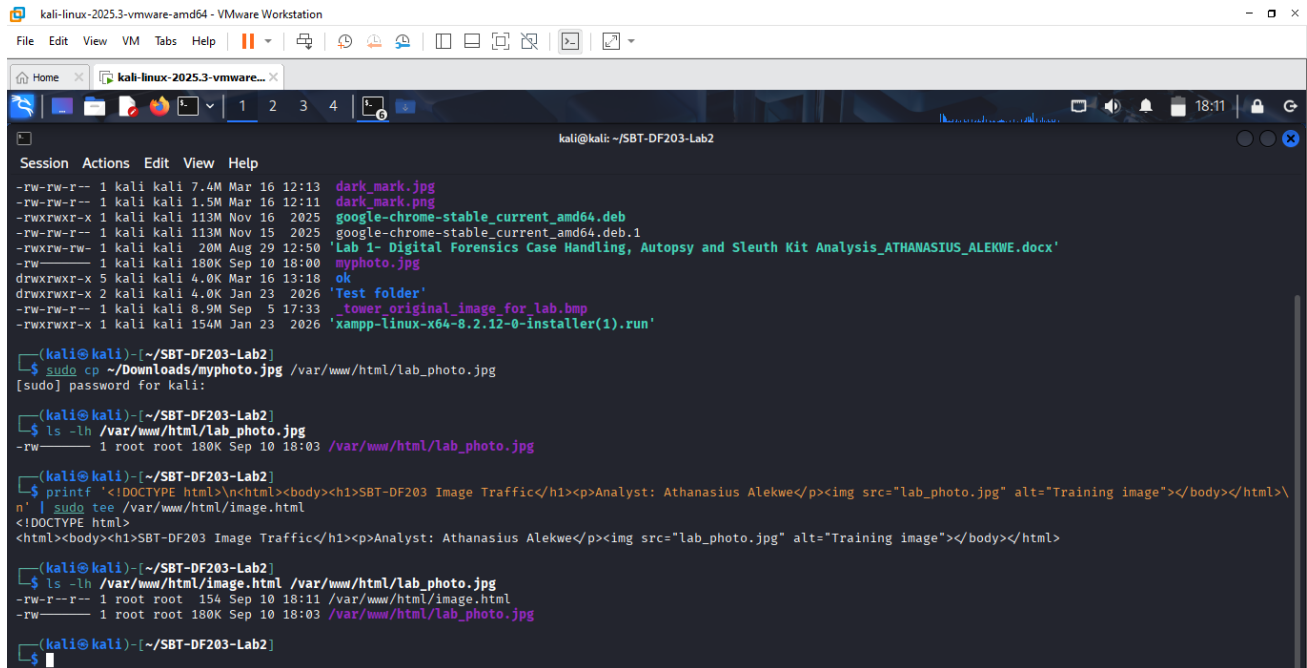
```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh ~/Downloads
total 1.1G
-rw-rw-r-- 1 kali kali 14K Aug 28 18:05 autopsy
-rw-rw-r-- 1 kali kali 14K Aug 28 19:41 'autopsy (1)'.
drwxrwxr-x 3 kali kali 4.0K Dec 21 2025 burpsuite
-rw-rw-r-- 1 kali kali 613M Dec 21 2025 burpsuite_pro_v2025.12.2.jar
-rw-rw-r-- 1 kali kali 939 May 23 15:01 cacert.der
-rw-rw-r-- 1 kali kali 1.5M Aug 28 11:17 Ch01InChap01.dd
-rw-rw-r-- 1 kali kali 7.4M Mar 16 12:13 dark.mark.jpg
-rw-rw-r-- 1 kali kali 1.5M Mar 16 12:11 dark.mark.png
-rwxrwxr-x 1 kali kali 113M Nov 16 2025 google-chrome-stable_current_amd64.deb
-rw-rw-r-- 1 kali kali 113M Nov 15 2025 google-chrome-stable_current_amd64.deb.1
-rwxrwxr-x 1 kali kali 20M Aug 29 12:50 'Lab 1- Digital Forensics Case Handling, Autopsy and Sleuth Kit Analysis_ATHANASIUS_ALEKWE.docx'
-rw-rw-r-- 1 kali kali 180K Sep 10 18:00 myphoto.jpg
drwxrwxr-x 5 kali kali 4.0K Mar 16 13:18 ok
drwxrwxr-x 2 kali kali 4.0K Jan 23 2026 'Test folder'
-rw-rw-r-- 1 kali kali 8.9M Sep 5 17:33 'tower original image for lab.bmp'
-rwxrwxr-x 1 kali kali 154M Jan 23 2026 'xampp-linux-x64-8.2.12-0-installer(1).run'

(kali@kali)~[~/SBT-DF203-Lab2]
$ sudo cp ~/Downloads/myphoto.jpg /var/www/html/lab_photo.jpg
[sudo] password for kali:
(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/lab_photo.jpg
-rw-rw-r-- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg
(kali@kali)~[~/SBT-DF203-Lab2]
$
```

Fig 1.4 Choosing the image and copying to Apache's web folder.

Explanation

This screenshot records the authorised JPEG selected for the exercise and the step used to place it in Apache's web directory. Renaming the file as lab_photo.jpg gave the webpage a simple and consistent image reference.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
-rw-rw-r-- 1 kali kali 7.4M Mar 16 12:13 dark_mark.jpg
-rw-rw-r-- 1 kali kali 1.5M Mar 16 12:11 dark_mark.png
-rwxrwxr-x 1 kali kali 113M Nov 16 2025 google-chrome-stable_current_amd64.deb
-rwxrwxr-x 1 kali kali 113M Nov 15 2025 google-chrome-stable_current_amd64.deb.1
-rwxrwxr-- 1 kali kali 20M Aug 29 12:50 'Lab 1- Digital Forensics Case Handling, Autopsy and Sleuth Kit Analysis_ATHANASIOS_ALEKWE.docx'
-rw-rw-r-- 1 kali kali 180K Sep 10 18:00 myphoto.jpg
drwxrwxr-x 5 kali kali 4.0K Mar 16 13:18 ok
drwxrwxr-x 2 kali kali 4.0K Jan 23 2026 'Test folder'
-rw-rw-r-- 1 kali kali 8.9M Sep 5 17:33 _tower_original_image_for_lab.bmp
-rwxrwxr-x 1 kali kali 154M Jan 23 2026 'xampp-linux-x64-8.2.12-0-installer(1).run'

(kali@kali)~[~/SBT-DF203-Lab2]
$ sudo cp ~/Downloads/myphoto.jpg /var/www/html/lab_photo.jpg
[sudo] password for kali:

(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/lab_photo.jpg
-rw-rw-r-- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: Athanasios Alekwe</p></body></html>\n' | sudo tee /var/www/html/image.html
!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: Athanasios Alekwe</p></body></html>

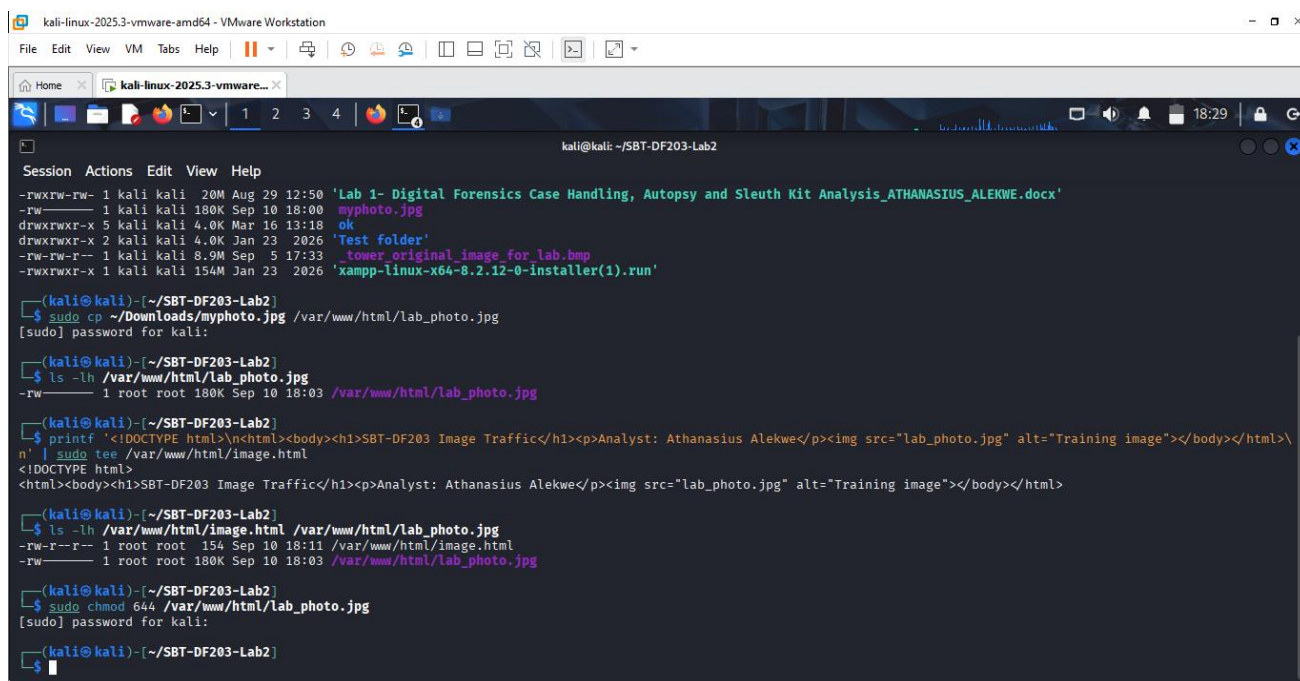
(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 154 Sep 10 18:11 /var/www/html/image.html
-rw-rw-r-- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2]
$
```

Fig 1.5 Creating the HTML page and verifying both files.

Explanation

The figure shows the creation of image.html with the analyst name and an element referencing lab_photo.jpg. Both source objects were then checked in the Apache web directory before traffic acquisition.



```
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
-rwxrwxrwx- 1 kali kali 20M Aug 29 12:50 'Lab 1- Digital Forensics Case Handling, Autopsy and Sleuth Kit Analysis_ATHANASIUS_ALEKWE.docx'
-rw- 1 kali kali 180K Sep 10 18:00 myphoto.jpg
drwxrwxr-x 5 kali kali 4.0K Mar 16 13:18 ok
drwxrwxr-x 2 kali kali 4.0K Jan 23 2026 'Test folder'
-rw-rw-r-- 1 kali kali 8.9M Sep 5 17:33 'tower original image for lab.bmp'
-rwxrwxr-x 1 kali kali 154M Jan 23 2026 'xampp-linux-x64-8.2.12-0-installer(1).run'

(kali@kali)~[~/SBT-DF203-Lab2]
$ sudo cp ~/Downloads/myphoto.jpg /var/www/html/lab_photo.jpg
[sudo] password for kali:

(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/lab_photo.jpg
-rw- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: Athanasius Alekwe</p></body></html>\n' | sudo tee /var/www/html/image.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: Athanasius Alekwe</p></body></html>

(kali@kali)~[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 154 Sep 10 18:11 /var/www/html/image.html
-rw- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg

(kali@kali)~[~/SBT-DF203-Lab2]
$ sudo chmod 644 /var/www/html/lab_photo.jpg
[sudo] password for kali:

(kali@kali)~[~/SBT-DF203-Lab2]
$
```

Fig 1.6 Changing the permission of the image file.

Explanation

This screenshot records the permission adjustment applied when the image initially failed to display correctly. After normal read permissions were set, Apache was able to serve the image successfully.

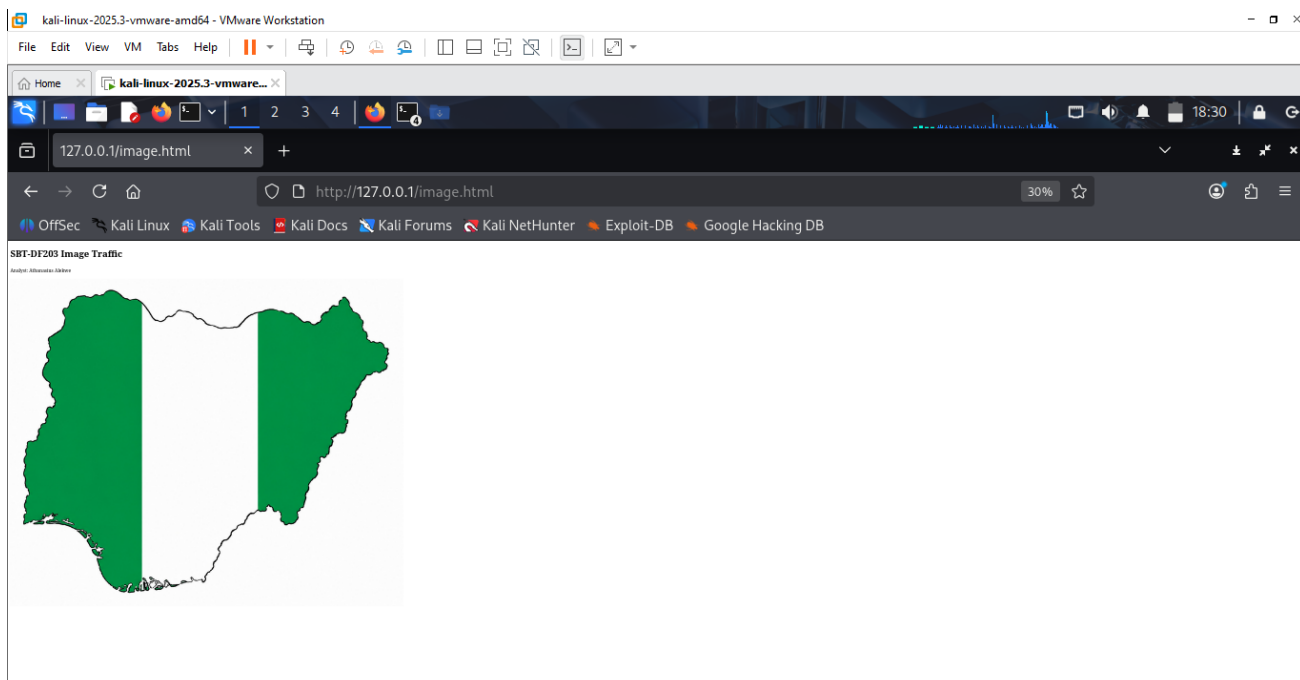


Fig 1.7 Webpage embedded image.

Explanation

This figure demonstrates that the locally hosted HTML page was accessible through

http://127.0.0.1/image.html. It confirms that the page content and analyst information were being served by Apache.

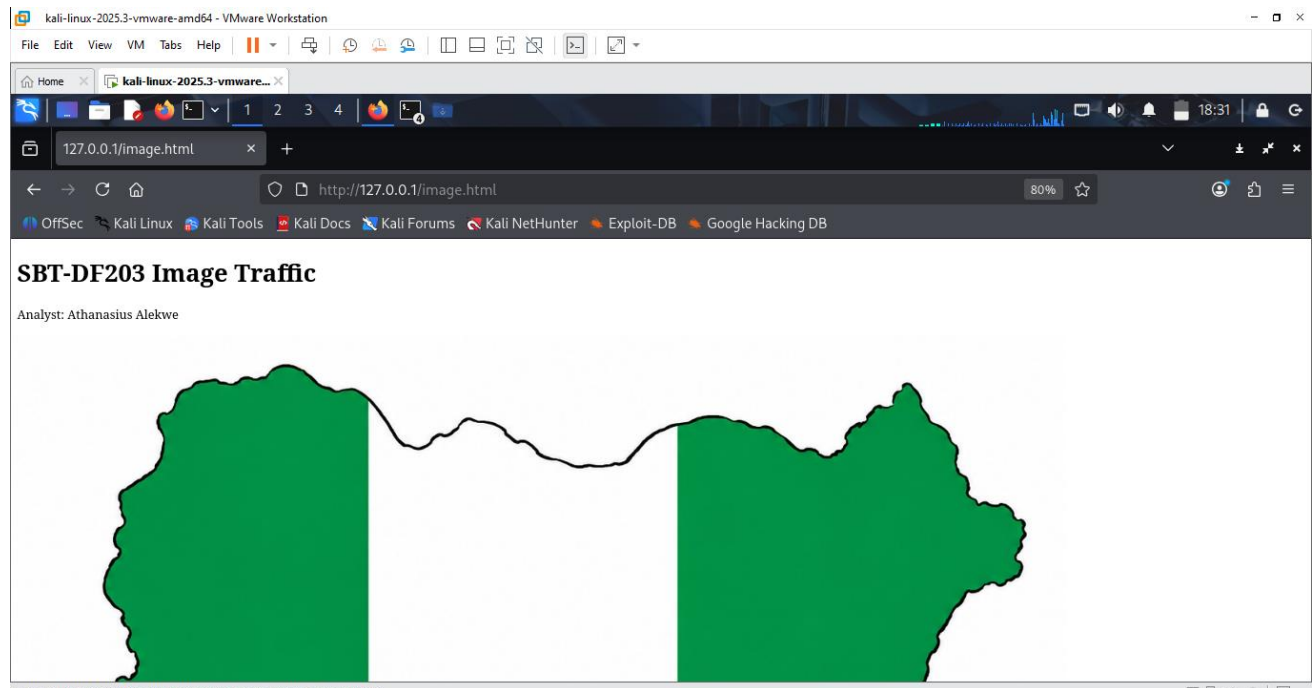


Fig 1.8 Webpage embedded image page 1.

Explanation

The screenshot confirms that the Nigeria JPEG loaded successfully as an embedded object within the webpage. This established that the HTML-to-image reference was working before the assessed packet capture began.

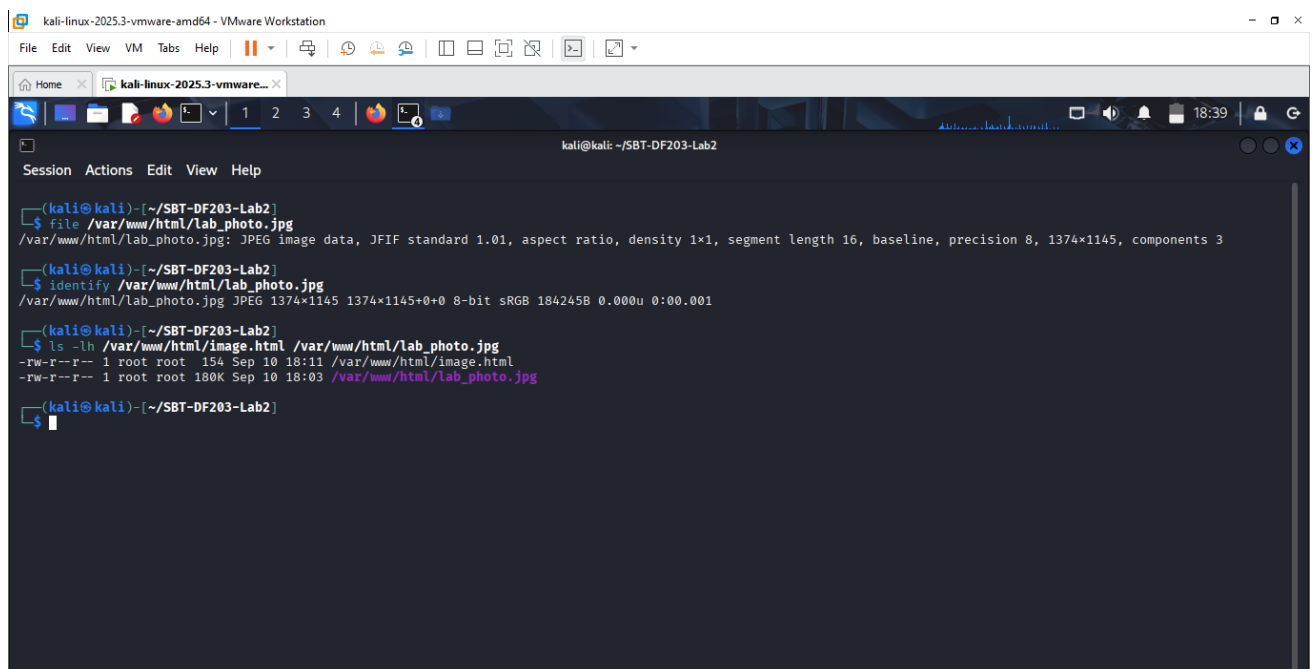
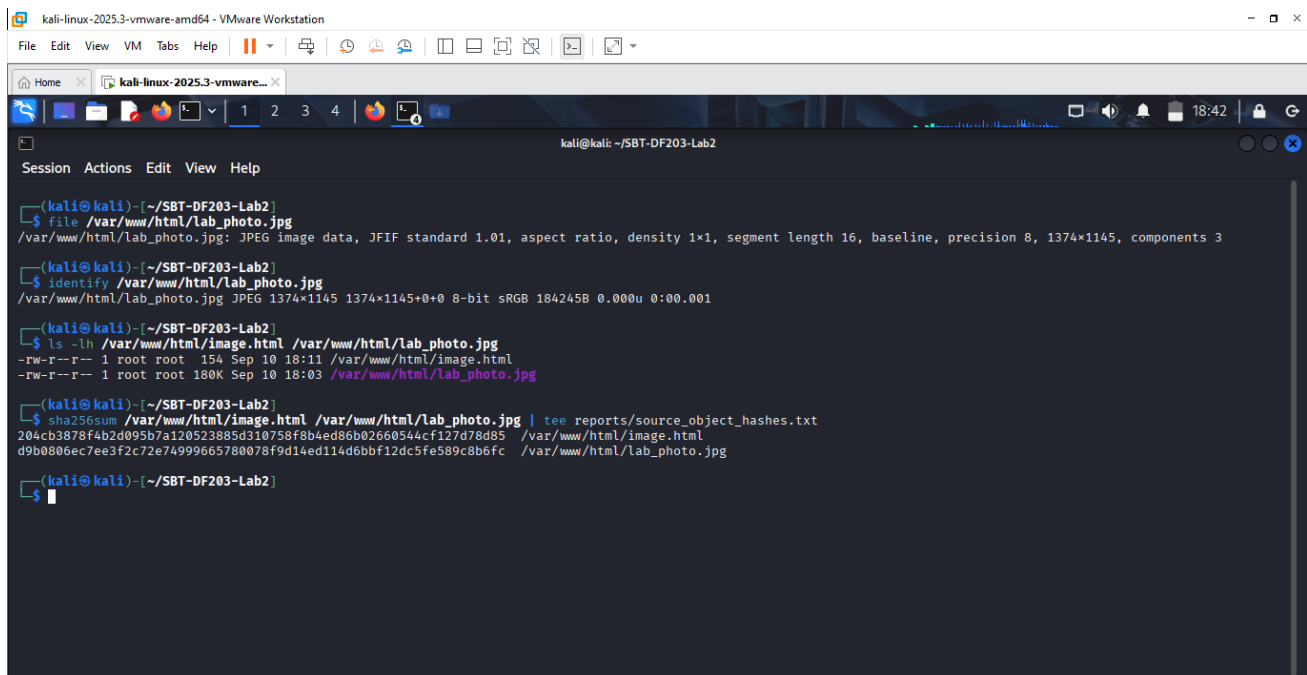


Fig 1.9 image file details.

File: /var/www/html/lab_photo.jpg
Type: JPEG/JFIF
Dimensions: 1374 × 1145 pixels
Exact image size: 182,245 bytes
Displayed disk size: about 180 KB
HTML file size: 154 bytes

Explanation

This figure records the JPEG/JFIF format and the image dimensions of 1374 × 1145 pixels, together with the HTML file information. These source characteristics provided a baseline for comparing the object recovered later from the network capture.



```
kali@kali: ~/SBT-DF203-Lab2
$ file /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 1374x1145, components 3

(kali@kali)~/SBT-DF203-Lab2$ identify /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg JPEG 1374x1145 1374x1145+0+0 8-bit sRGB 184245B 0.000u 0:00.001

(kali@kali)~/SBT-DF203-Lab2$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 154 Sep 10 18:11 /var/www/html/image.html
-rw-r--r-- 1 root root 180K Sep 10 18:03 /var/www/html/lab_photo.jpg

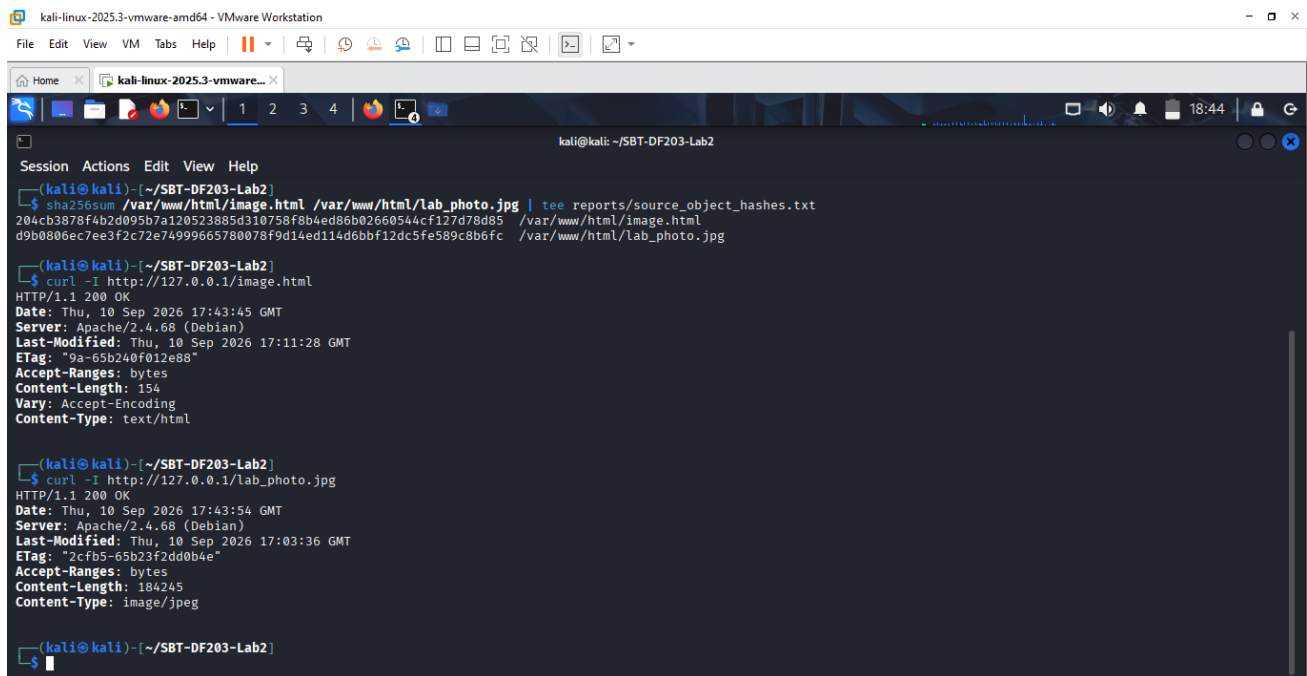
(kali@kali)~/SBT-DF203-Lab2$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
204cb3878f4b2d095b7a120523885d310758f8b4ed86b02660544cf127d78d85 /var/www/html/image.html
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc /var/www/html/lab_photo.jpg

(kali@kali)~/SBT-DF203-Lab2$
```

Fig 2.0 Generating the baseline SHA-256 hashes.

Explanation

This screenshot records the SHA-256 values calculated for the source HTML and JPEG before the main analysis. The image hash later became the reference value used to test whether the recovered JPEG had changed during transfer or extraction.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali) ~/SBT-DF203-Lab2
$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
204cb3878f4b2d095b7a120523885d310758f8b4ed86b02660544cf127d78d85 /var/www/html/image.html
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc /var/www/html/lab_photo.jpg
(kali@kali) ~/SBT-DF203-Lab2
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 17:43:45 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 17:11:28 GMT
ETag: "9a-65b240f012e88"
Accept-Ranges: bytes
Content-Length: 154
Vary: Accept-Encoding
Content-Type: text/html
(kali@kali) ~/SBT-DF203-Lab2
$ curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 17:43:54 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 17:03:36 GMT
ETag: "2cfb5-65b23f2dd0b4e"
Accept-Ranges: bytes
Content-Length: 184245
Content-Type: image/jpeg
(kali@kali) ~/SBT-DF203-Lab2
$
```

Fig 2.1 Verifying both HTTP objects.

image.html:	HTTP 200 OK
HTML Content-Length:	154 bytes
HTML Content-Type:	text/html
lab_photo.jpg:	HTTP 200 OK
Image Content-Length:	182,425 bytes
Image Content-Type:	image/jpeg
Apache server:	Apache/2.4.68 (Debian)

Explanation

The figure shows that Apache returned successful HTTP responses for both image.html and lab_photo.jpg. It also records the MIME types and Content-Length information used as baseline HTTP metadata.

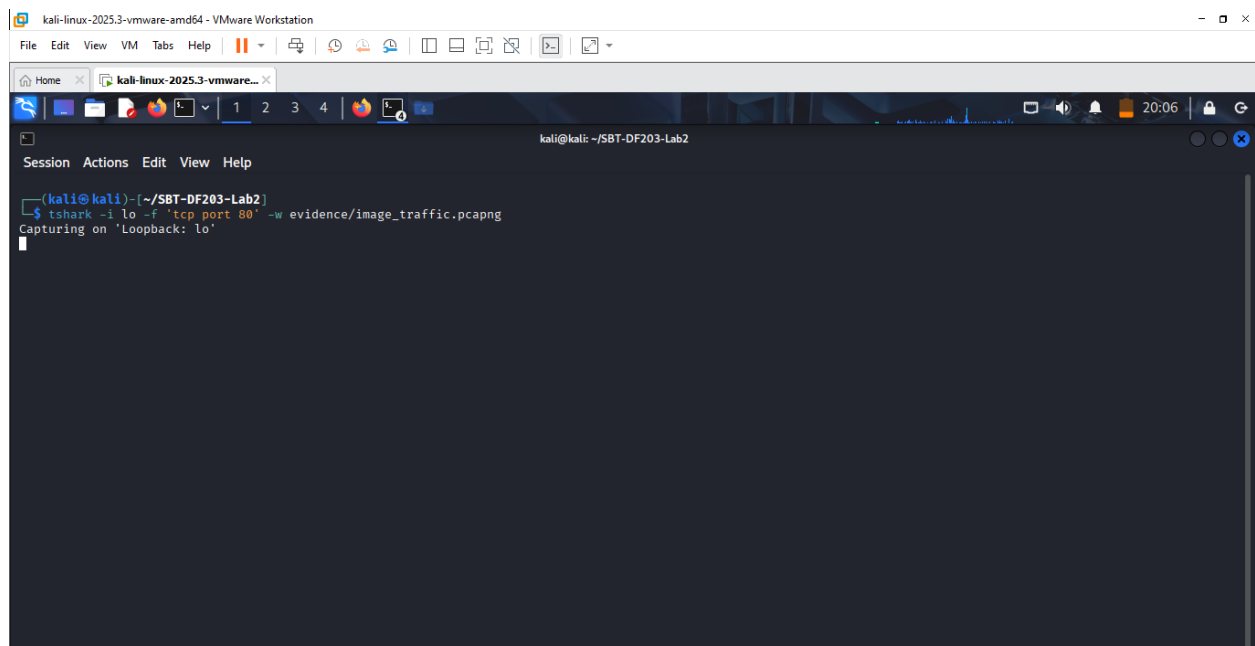


Fig 2.2 Capturing on 'Loopback'.

Explanation

This screenshot confirms that TShark successfully began monitoring the lo interface with traffic restricted to TCP port 80. This ensured that the capture focused on the locally generated plaintext HTTP traffic.

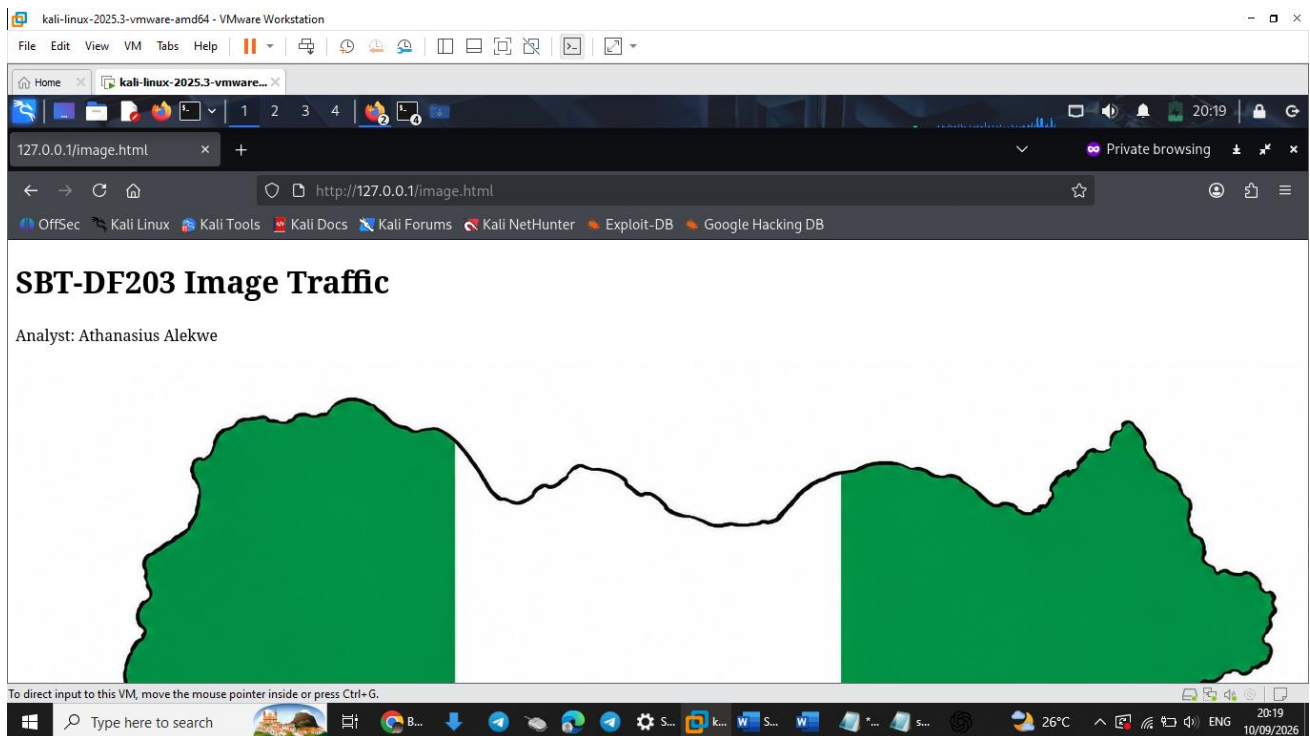


Fig 2.3 Capturing the traffic.

Explanation

The figure shows the active TShark session used while Firefox accessed the webpage. Keeping the capture active during the page load allowed both the HTML request and the browser-generated image request to be recorded.

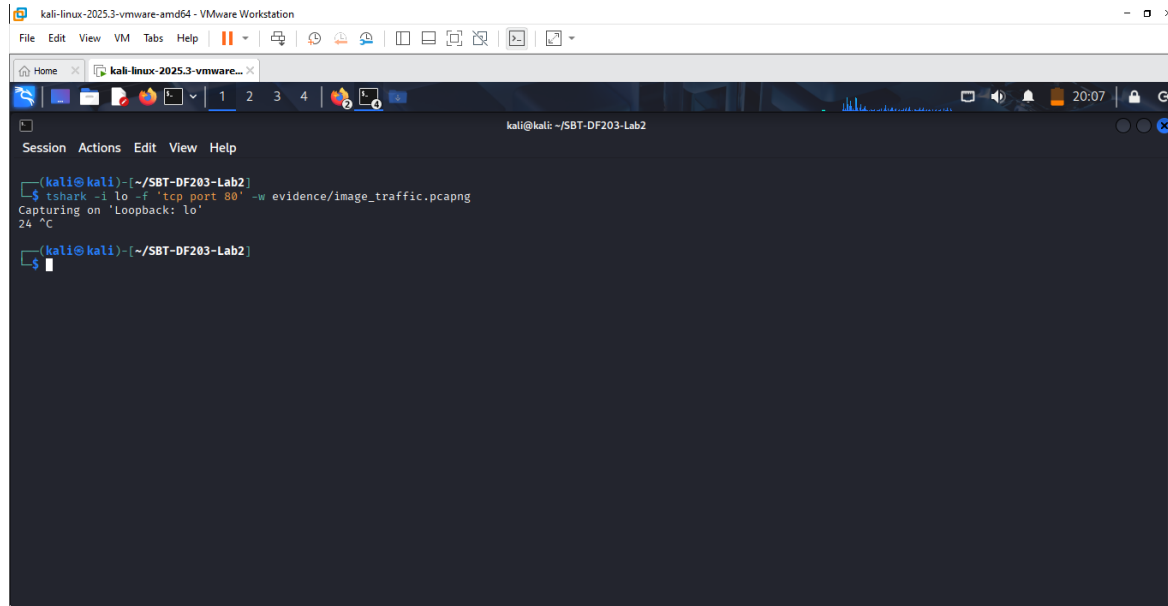


Fig 2.4 24 packets captured.

Explanation

This screenshot documents the point at which the browser capture was stopped after the webpage and embedded image had loaded. For the final report, the analysis values should be taken from the final preserved image_traffic.pcapng capture used for the request, response and TCP calculations.

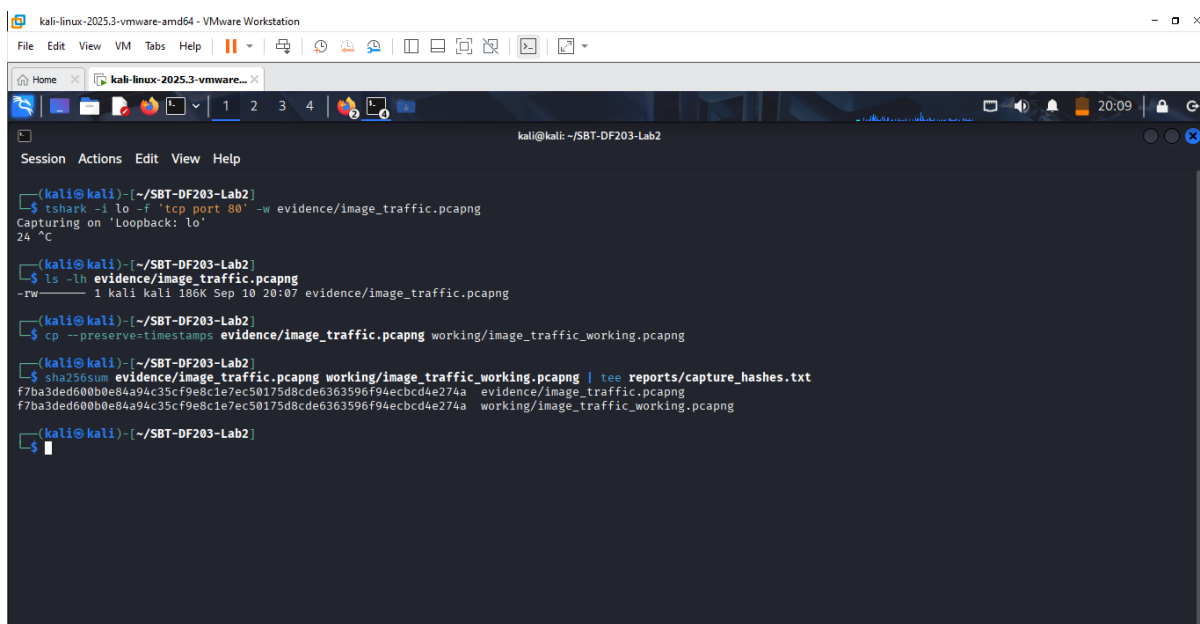
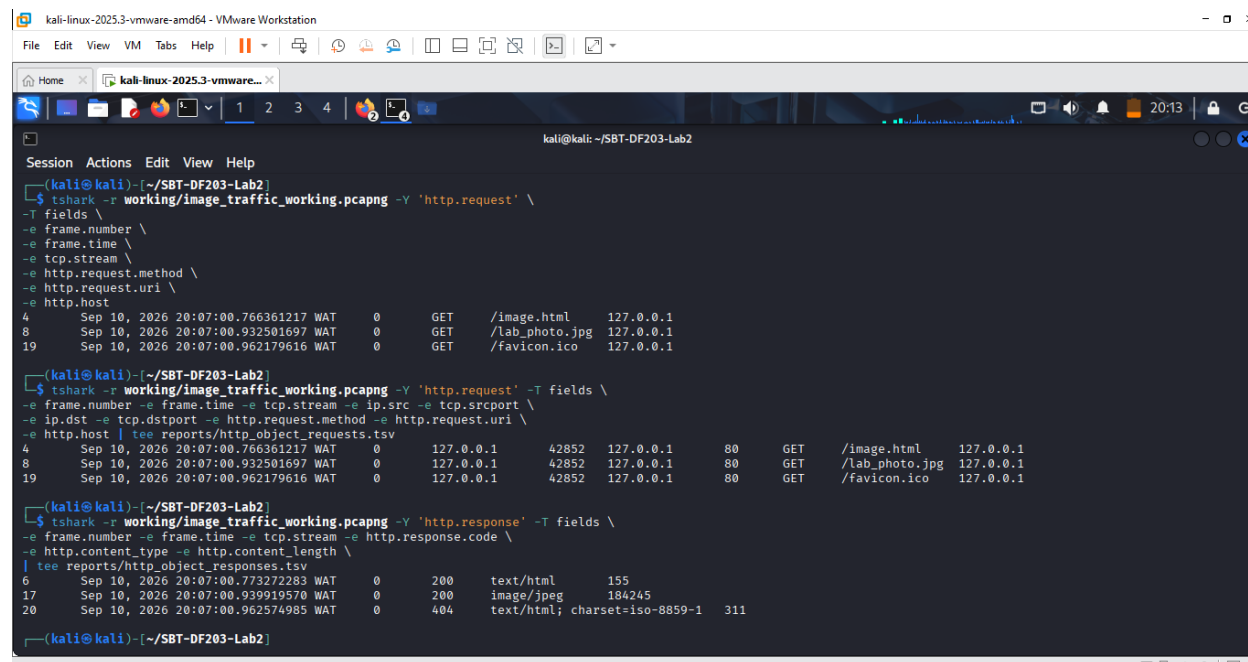


Fig 2.5 Browser capture and pcap hashes.

Explanation

The screenshot shows the original PCAPNG being copied to the working directory and both files being hashed. Their matching SHA-256 values demonstrate that the working capture was an exact copy of the preserved evidence.



```
kali@kali: ~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-T fields \
-e frame.number \
-e frame.time \
-e tcp.stream \
-e http.request.method \
-e http.request.uri \
-e http.host
4   Sep 10, 2026 20:07:00.766361217 WAT  0      GET      /image.html 127.0.0.1
8   Sep 10, 2026 20:07:00.932501697 WAT  0      GET      /lab_photo.jpg 127.0.0.1
19  Sep 10, 2026 20:07:00.962179616 WAT  0      GET      /favicon.ico 127.0.0.1

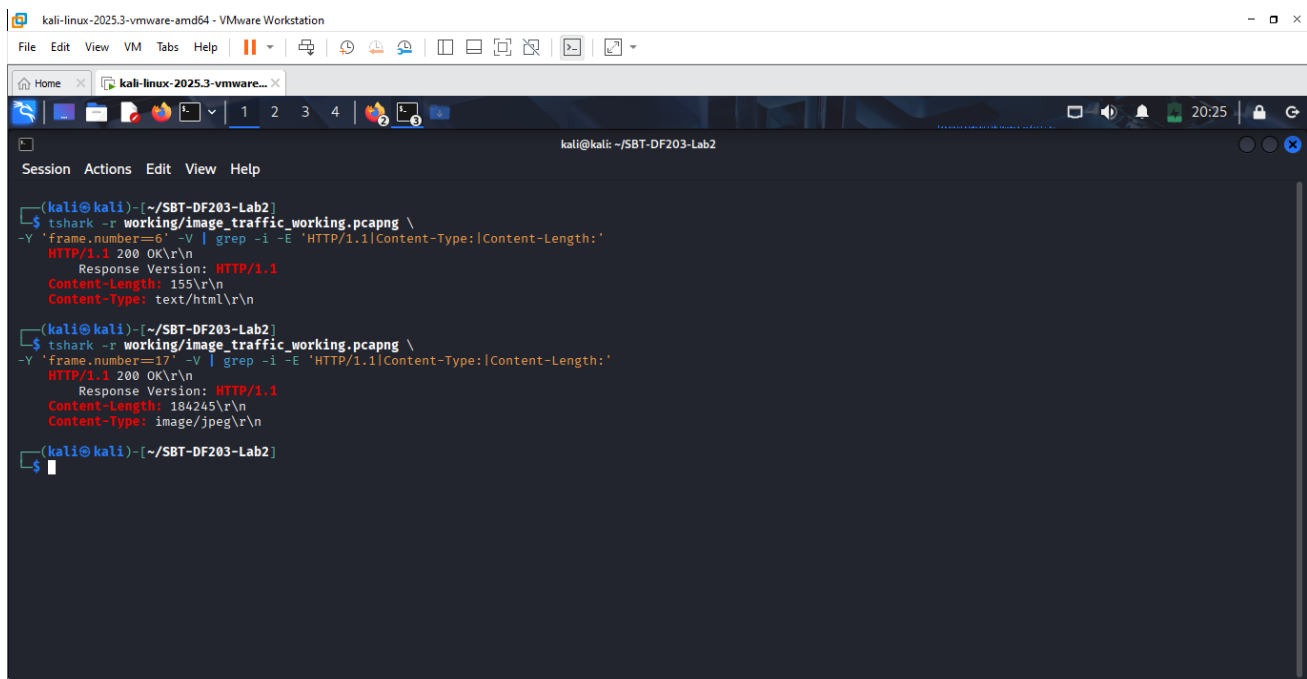
(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e http.request.method -e http.request.uri \
-e http.host | tee reports/http_object_requests.tsv
4   Sep 10, 2026 20:07:00.766361217 WAT  0      127.0.0.1  42852  127.0.0.1  80      GET      /image.html 127.0.0.1
8   Sep 10, 2026 20:07:00.932501697 WAT  0      127.0.0.1  42852  127.0.0.1  80      GET      /lab_photo.jpg 127.0.0.1
19  Sep 10, 2026 20:07:00.962179616 WAT  0      127.0.0.1  42852  127.0.0.1  80      GET      /favicon.ico 127.0.0.1

(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e tcp.stream -e http.response.code \
-e http.content_type -e http.content_length \
| tee reports/http_object_responses.tsv
6   Sep 10, 2026 20:07:00.773272283 WAT  0      200      text/html 155
17  Sep 10, 2026 20:07:00.939919570 WAT  0      200      image/jpeg 184245
20  Sep 10, 2026 20:07:00.962574985 WAT  0      404      text/html; charset=iso-8859-1 311
```

Fig 2.6 Recording the HTTP request/response.

Explanation

This figure identifies the individual requests created by Firefox, including /image.html, /lab_photo.jpg and /favicon.ico. The corresponding response information shows successful 200 OK responses for the HTML and JPEG and a 404 Not Found response for the favicon.



```
kali@kali: ~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng \
-Y 'frame.number==6' -V | grep -i -E 'HTTP/1.1|Content-Type:|Content-Length:'
HTTP/1.1 200 OK\r\n
  Response Version: HTTP/1.1
  Content-Length: 155\r\n
  Content-Type: text/html\r\n

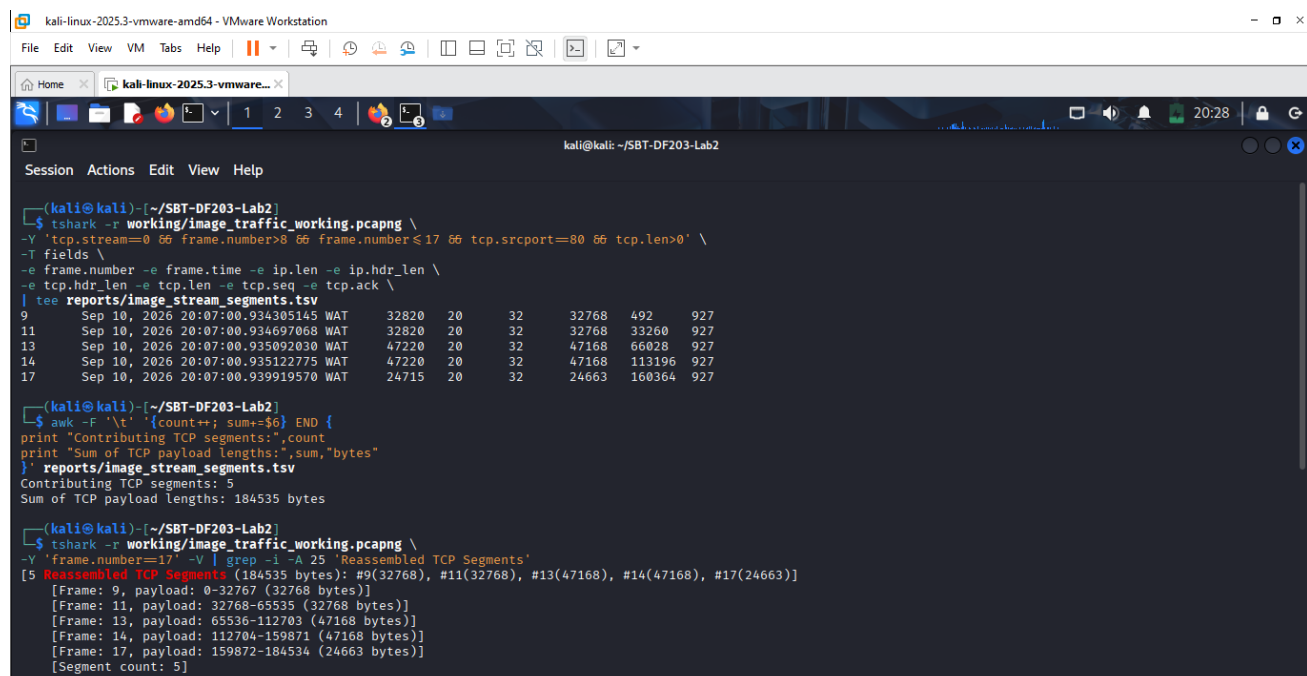
(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng \
-Y 'frame.number==17' -V | grep -i -E 'HTTP/1.1|Content-Type:|Content-Length:'
HTTP/1.1 200 OK\r\n
  Response Version: HTTP/1.1
  Content-Length: 184245\r\n
  Content-Type: image/jpeg\r\n

(kali@kali)~/SBT-DF203-Lab2
$
```

Fig 2.7 Capturing HTTP headers.

Explanation

The screenshot exposes the HTTP response metadata for the transferred objects. Of particular importance, the JPEG response is identified as image/jpeg with a Content-Length of 184,245 bytes.



```
kali@kali: ~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng \
-Y 'tcp.stream==0 && frame.number>8 && frame.number<=17 && tcp.srcport==80 && tcp.len>0' \
-T fields \
-e frame.number -e frame.time -e ip.len -e ip.hdr_len \
-e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
| tee reports/image_stream_segments.tsv
9  Sep 10, 2026 20:07:00.934305145 WAT 32820 20 32 32768 492 927
11 Sep 10, 2026 20:07:00.934697068 WAT 32820 20 32 32768 33260 927
13 Sep 10, 2026 20:07:00.935092030 WAT 47220 20 32 47168 66028 927
14 Sep 10, 2026 20:07:00.935122775 WAT 47220 20 32 47168 113196 927
17 Sep 10, 2026 20:07:00.939919570 WAT 24715 20 32 24663 160364 927

(kali@kali)~/SBT-DF203-Lab2
$ awk -F '\t' '{count++; sum+=$6} END {
print "Contributing TCP segments:",count
print "Sum of TCP payload lengths:",sum,"bytes"
}' reports/image_stream_segments.tsv
Contributing TCP segments: 5
Sum of TCP payload lengths: 184535 bytes

(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng \
-Y 'frame.number==17' -V | grep -i -A 25 'Reassembled TCP Segments'
[5 Reassembled TCP Segments (184535 bytes): #9(32768), #11(32768), #13(47168), #14(47168), #17(24663)]
[Frame: 9, payload: 0-32767 (32768 bytes)]
[Frame: 11, payload: 32768-65535 (32768 bytes)]
[Frame: 13, payload: 65536-112703 (47168 bytes)]
[Frame: 14, payload: 112704-159871 (47168 bytes)]
[Frame: 17, payload: 159872-184534 (24663 bytes)]
[Segment count: 5]
```

Fig 2.8 Capturing tshark details.

Explanation

This figure shows the TCP-level values extracted from the image-transfer portion of the capture, including IP length, TCP header length, TCP payload length, sequence numbers and acknowledgements. These values were used to establish how the JPEG response was distributed across the TCP stream.

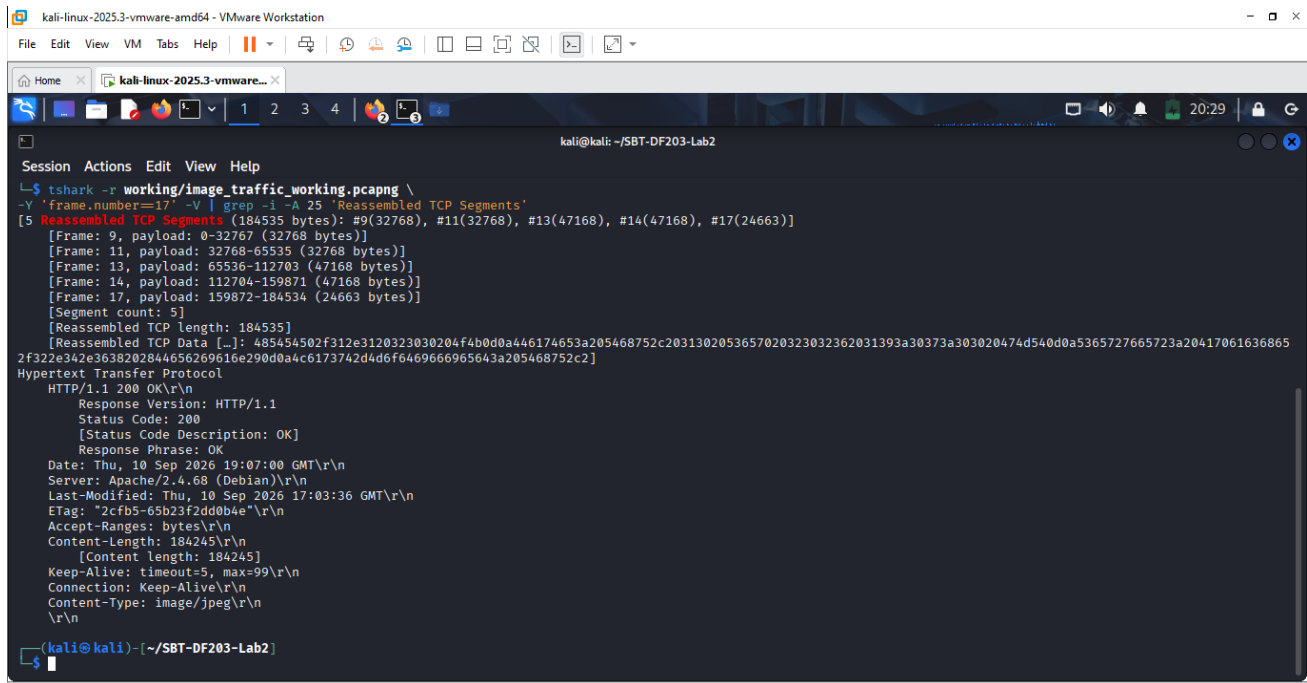


Fig 2.9. Capturing tshark details page 1.

Frame	IP length	IP header	TCP header	TCP payload (tcp.len)
9	32,820	20	32	32,768
11	32,820	20	32	32,768
13	47,220	20	32	47,168
14	47,220	20	32	47,168
17	24,715	20	32	24,663

The total TCP payload is:
 32,768 + 32,768 + 47,168 + 47,168 + 24,663 = 184,535 bytes

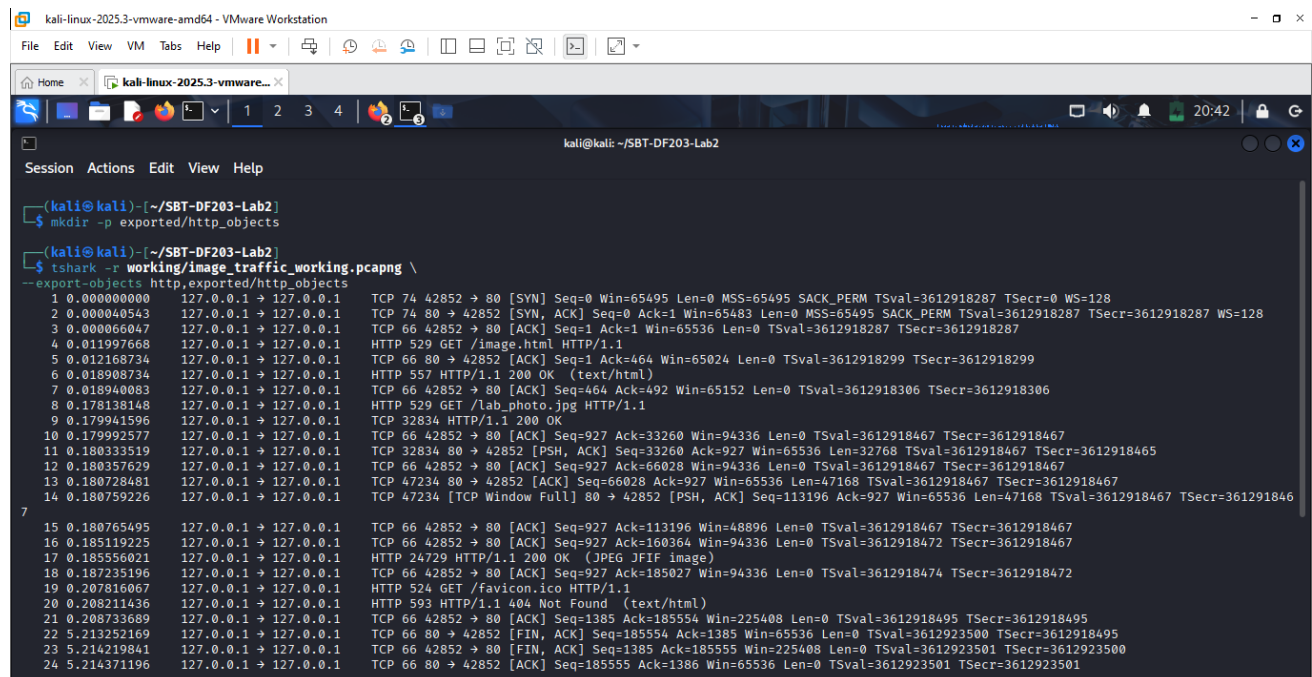
TShark independently confirms:
 Reassembled TCP length: 184535
 Segment count: 5

The HTTP response says:
 Content-Length: 184245
 Content-Type: image/jpeg
 So we have an important size relationship:
 Reassembled TCP payload 184,535 bytes
 JPEG HTTP Content-Length -184,245 bytes
 HTTP response headers 290 bytes

The 184,535 bytes of reassembled TCP application payload contain both the HTTP response headers and the JPEG data. Of that total, 184,245 bytes are the actual JPEG object, leaving 290 bytes for the HTTP response header block.

Explanation

This screenshot provides the key reassembly evidence: five TCP segments combined to produce 184,535 bytes of TCP application payload. Comparing this with the 184,245-byte JPEG body shows that the remaining 290 bytes belong to the HTTP response header.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab2
$ mkdir -p exported/http_objects
(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng \
--export-objects http,exported/http_objects
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 42852 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3612918287 TSecr=0 WS=128
2 0.000040543 127.0.0.1 → 127.0.0.1 TCP 74 80 → 42852 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3612918287 TSecr=3612918287 WS=128
3 0.000066047 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3612918287 TSecr=3612918287
4 0.011997668 127.0.0.1 → 127.0.0.1 HTTP 529 GET /image.html HTTP/1.1
5 0.012168734 127.0.0.1 → 127.0.0.1 TCP 66 80 → 42852 [ACK] Seq=1 Ack=464 Win=65024 Len=0 TSval=3612918299 TSecr=3612918299
6 0.018908734 127.0.0.1 → 127.0.0.1 HTTP 557 HTTP/1.1 200 OK (text/html)
7 0.018940083 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=464 Ack=492 Win=65152 Len=0 TSval=3612918306 TSecr=3612918306
8 0.178138148 127.0.0.1 → 127.0.0.1 HTTP 529 GET /lab_photo.jpg HTTP/1.1
9 0.179941596 127.0.0.1 → 127.0.0.1 TCP 32834 HTTP/1.1 200 OK
10 0.179992577 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=927 Ack=33260 Win=94336 Len=0 TSval=3612918467 TSecr=3612918467
11 0.180333519 127.0.0.1 → 127.0.0.1 TCP 32834 80 → 42852 [PSH, ACK] Seq=33260 Ack=927 Win=65536 Len=32768 TSval=3612918467 TSecr=3612918465
12 0.180357629 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=927 Ack=66028 Win=94336 Len=0 TSval=3612918467 TSecr=3612918467
13 0.180728481 127.0.0.1 → 127.0.0.1 TCP 47234 80 → 42852 [ACK] Seq=66028 Ack=927 Win=65536 Len=47168 TSval=3612918467 TSecr=3612918467
14 0.180759226 127.0.0.1 → 127.0.0.1 TCP 47234 [TCP Window Full] 80 → 42852 [PSH, ACK] Seq=113196 Ack=927 Win=65536 Len=47168 TSval=3612918467 TSecr=361291846
7
15 0.180765495 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=927 Ack=113196 Win=48896 Len=0 TSval=3612918467 TSecr=3612918467
16 0.185119225 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=927 Ack=160364 Win=94336 Len=0 TSval=3612918472 TSecr=3612918467
17 0.185556021 127.0.0.1 → 127.0.0.1 HTTP 24729 HTTP/1.1 200 OK (JPEG JFIF image)
18 0.187235196 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=927 Ack=185027 Win=94336 Len=0 TSval=3612918474 TSecr=3612918472
19 0.207816067 127.0.0.1 → 127.0.0.1 HTTP 524 GET /favicon.ico HTTP/1.1
20 0.208211436 127.0.0.1 → 127.0.0.1 HTTP 593 HTTP/1.1 404 Not Found (text/html)
21 0.208733689 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [ACK] Seq=1385 Ack=185554 Win=225408 Len=0 TSval=3612918495 TSecr=3612918495
22 5.213252169 127.0.0.1 → 127.0.0.1 TCP 66 80 → 42852 [FIN, ACK] Seq=185554 Ack=1385 Win=65536 Len=0 TSval=3612923500 TSecr=3612918495
23 5.214219841 127.0.0.1 → 127.0.0.1 TCP 66 42852 → 80 [FIN, ACK] Seq=1385 Ack=185555 Win=225408 Len=0 TSval=3612923501 TSecr=3612923500
24 5.214371196 127.0.0.1 → 127.0.0.1 TCP 66 80 → 42852 [ACK] Seq=185555 Ack=1386 Win=65536 Len=0 TSval=3612923501 TSecr=3612923501
```

Fig 3.0 Exporting the transferred JPEG.

Explanation

This figure shows TShark's HTTP object-export function being applied to the working PCAPNG. It demonstrates that the transferred resources could be reconstructed directly from the captured network traffic.

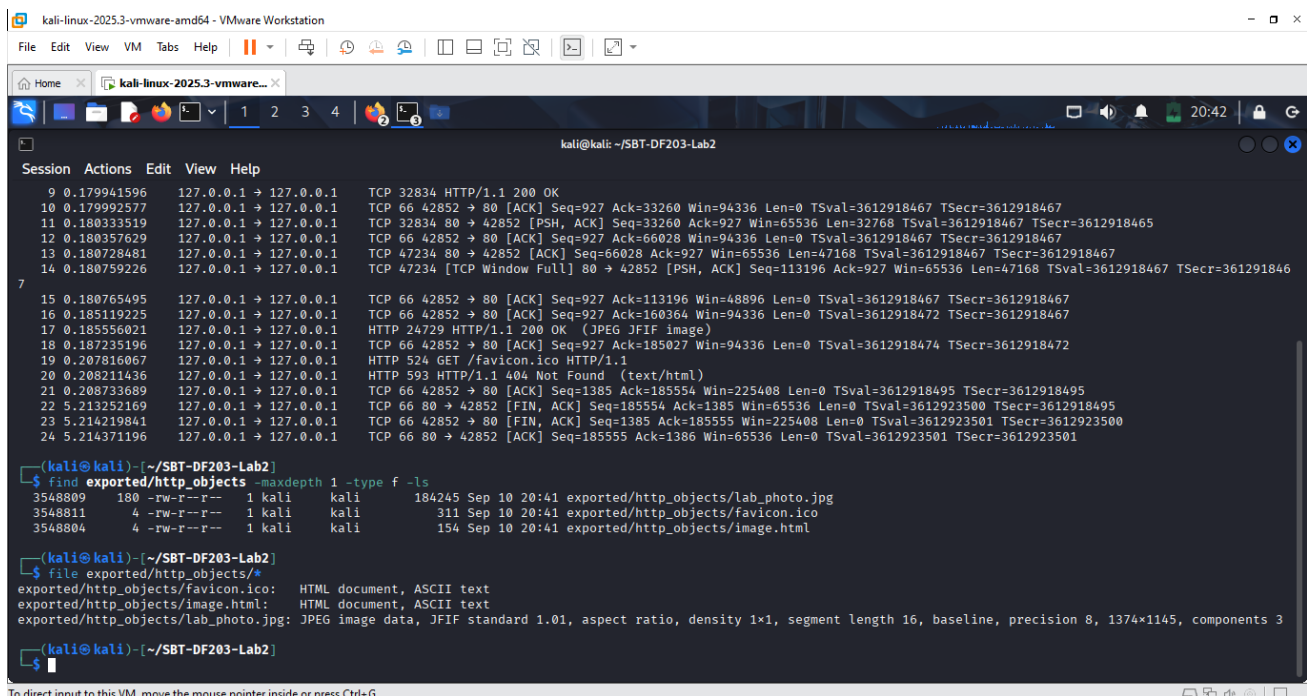


Fig 3.0.1 Exporting the transferred JPEG page 1.

Explanation

The screenshot lists the objects recovered by TShark, including image.html, lab_photo.jpg and the favicon response. The recovered JPEG was recognised as a valid JPEG/JFIF image with the expected dimensions.

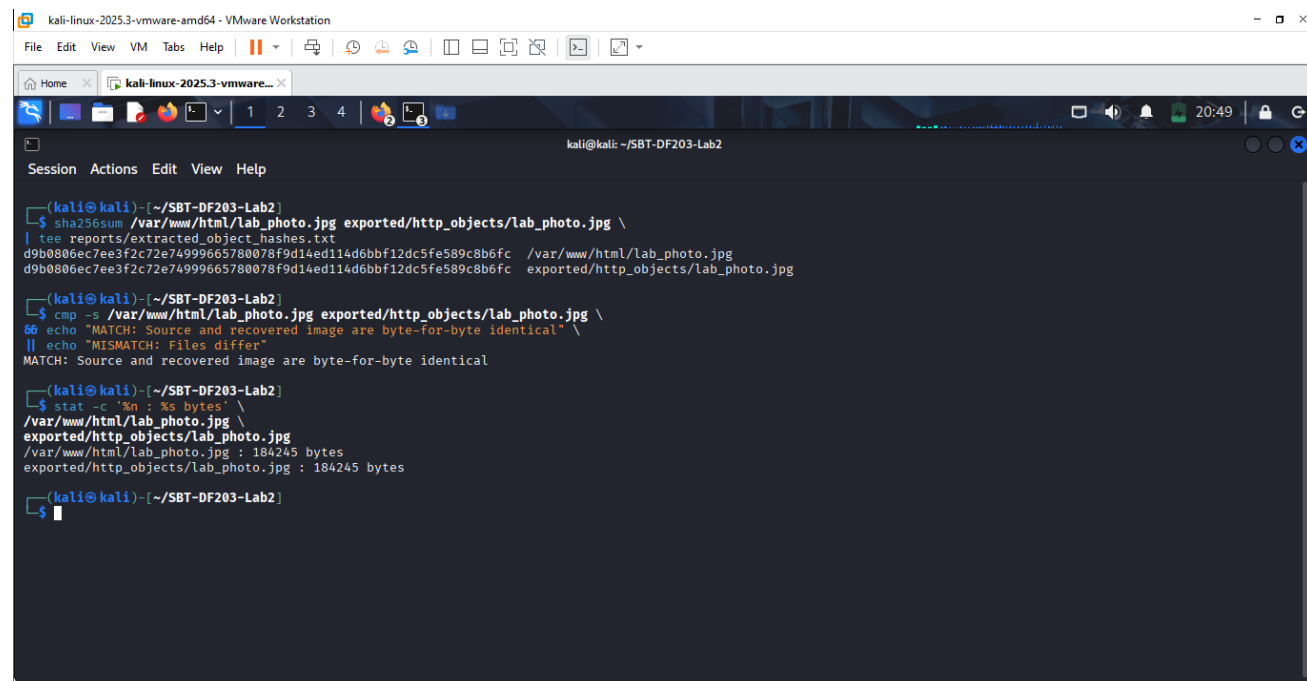


Fig 3.1 SHA-256 integrity test.

The SHA-256 values of the original and recovered JPEG are identical, demonstrating that the image extracted from the HTTP capture is byte-for-byte identical to the source object and was recovered without alteration.

Explanation

This figure compares the SHA-256 value of the source JPEG with the object recovered from the capture. The two hashes match, and the byte comparison confirms that the source and recovered files are identical.

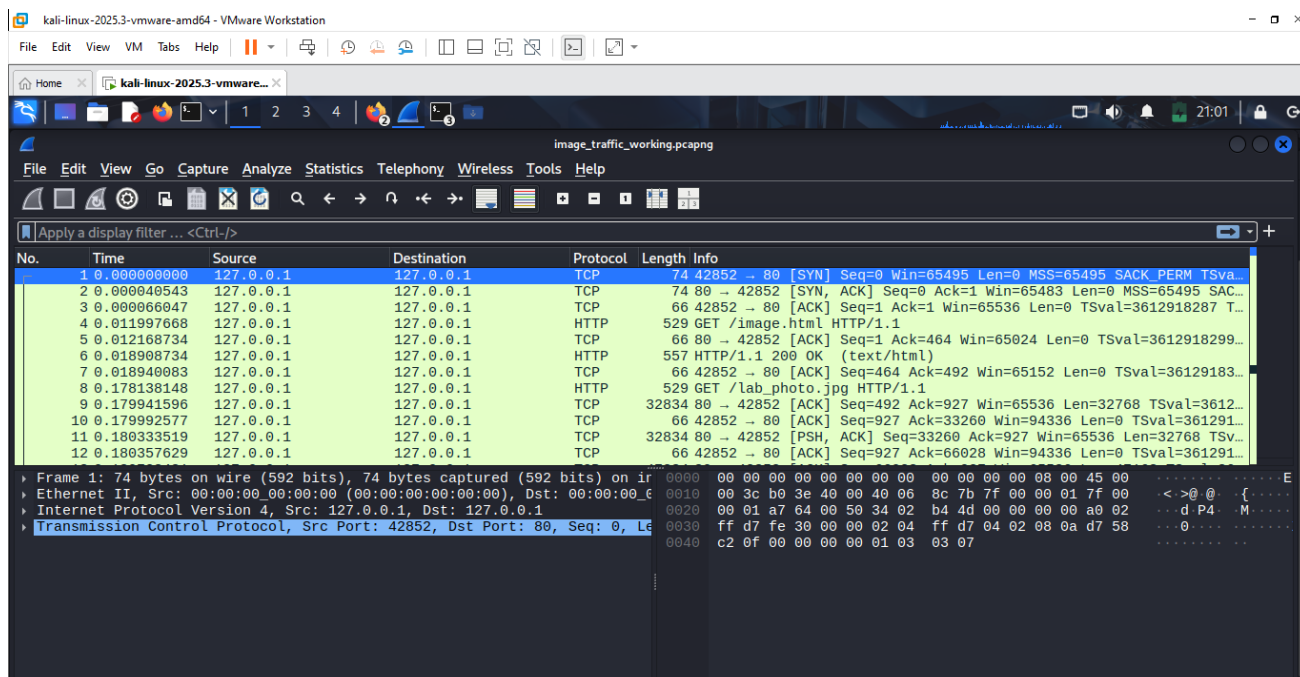


Fig 3.2 Opening Wireshark to perform manual.

Explanation

The screenshot shows the preserved working capture being opened in Wireshark for graphical analysis. This allowed the same HTTP objects to be inspected and exported manually rather than relying only on TShark.

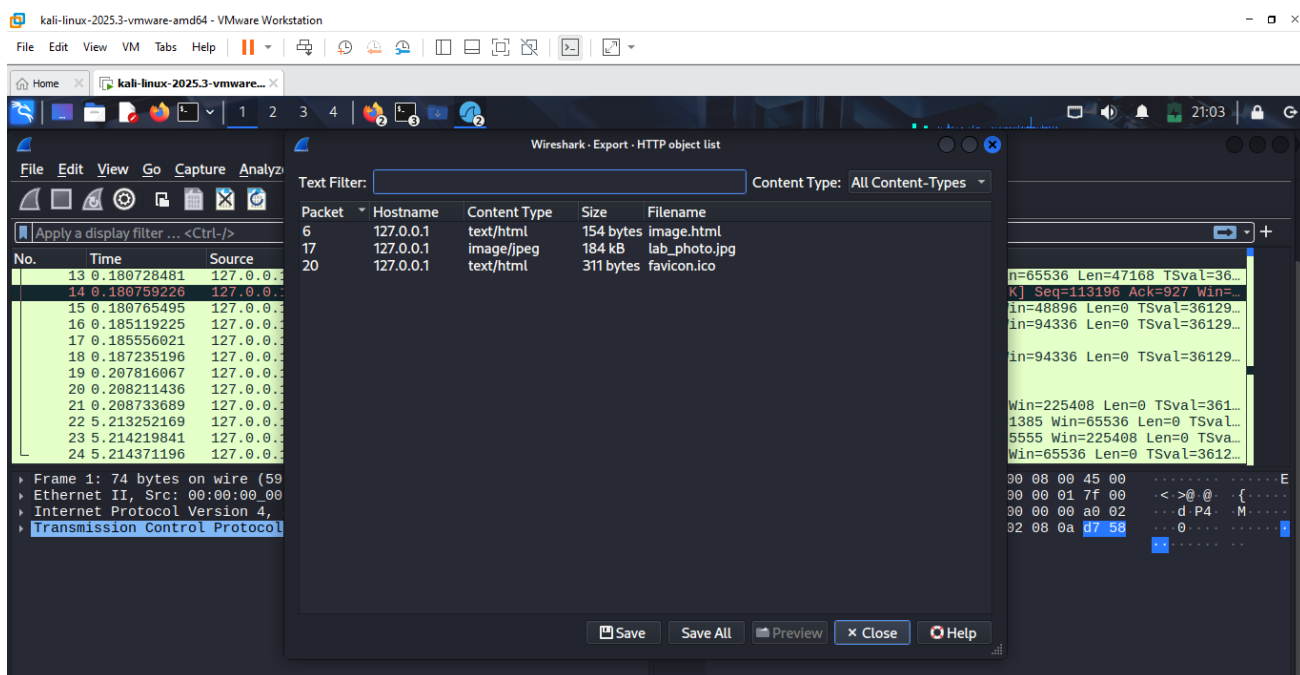


Fig 3.3 Opening Wireshark—Export--HTTP.

Explanation

This figure displays Wireshark's HTTP Export Objects window containing image.html, lab_photo.jpg and favicon.ico. It provides visual evidence that Wireshark reconstructed the transferred resources from the captured HTTP session.

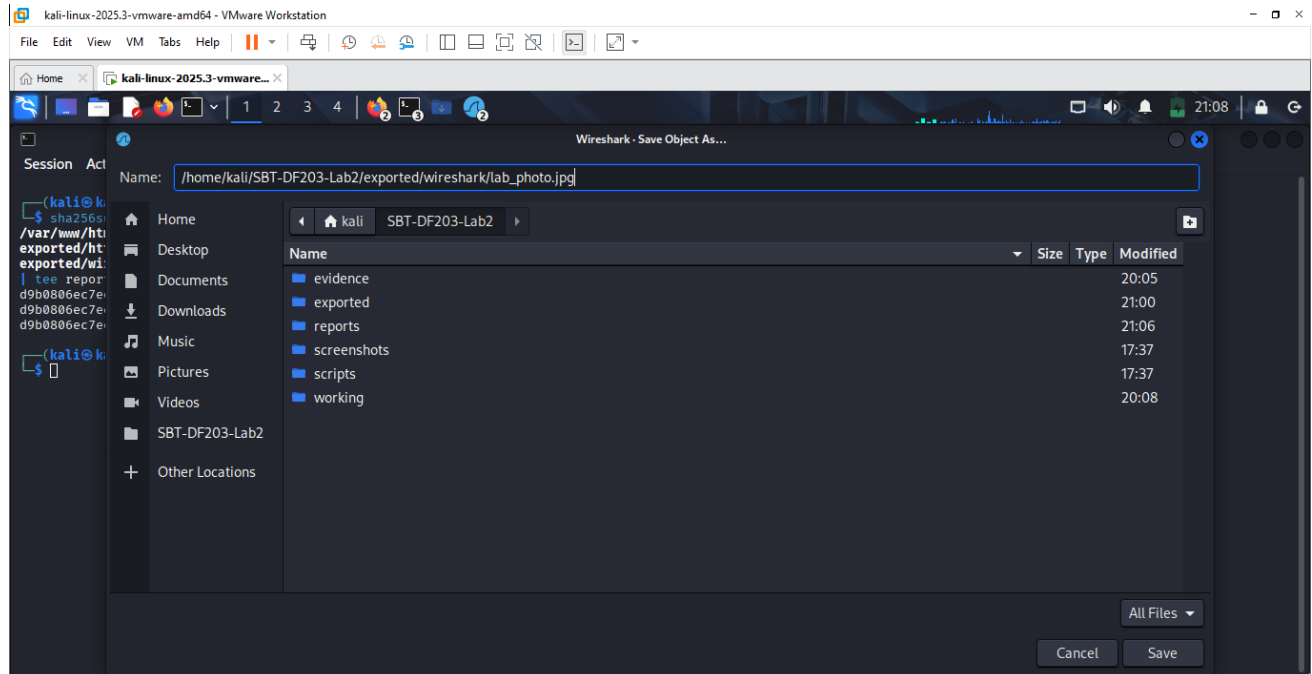


Fig 3.4 Saving to /SBT-DF203-Lab2/exported/wireshark/lab_photo.jpg

Explanation

The screenshot documents the manual saving of lab_photo.jpg into the designated exported/wireshark directory. Using a separate destination preserved a clear distinction between the TShark and Wireshark extraction methods.

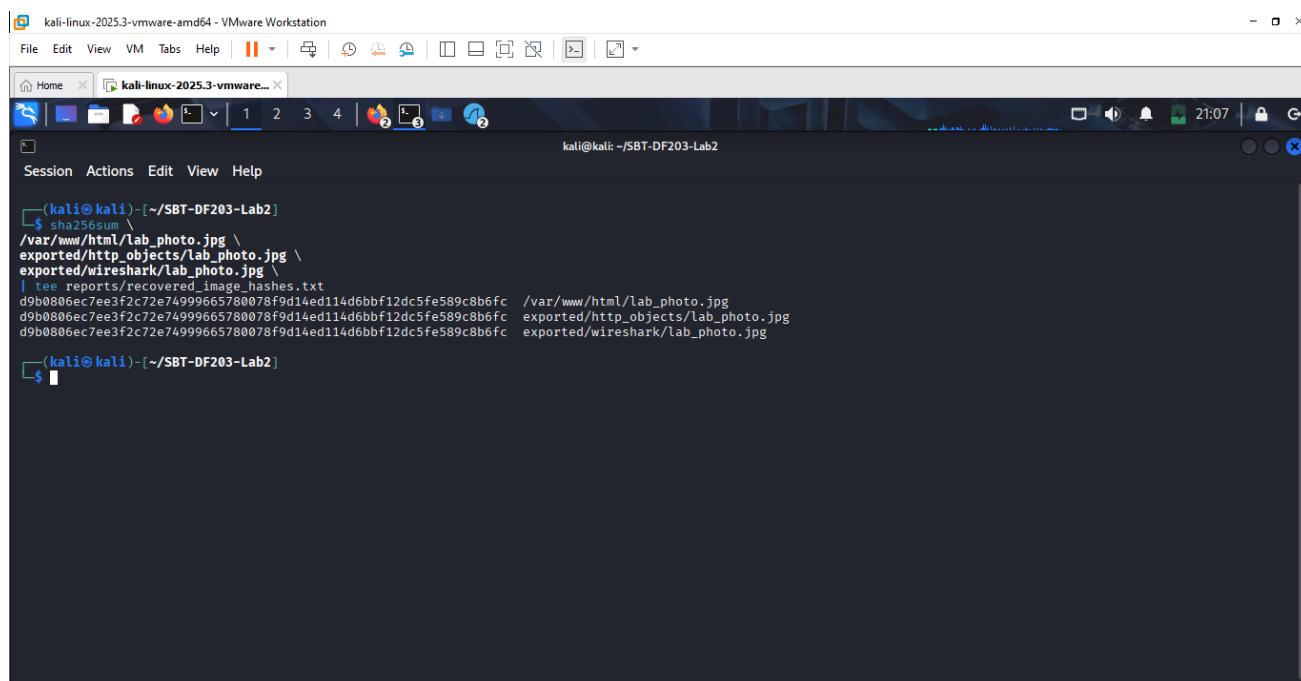


Fig 3.5 Verifying the GUI-exported version.

Explanation

This figure records the integrity check carried out after the Wireshark export. Matching hashes between the source and recovered copies show that the graphical extraction method also reproduced the original object without modification.

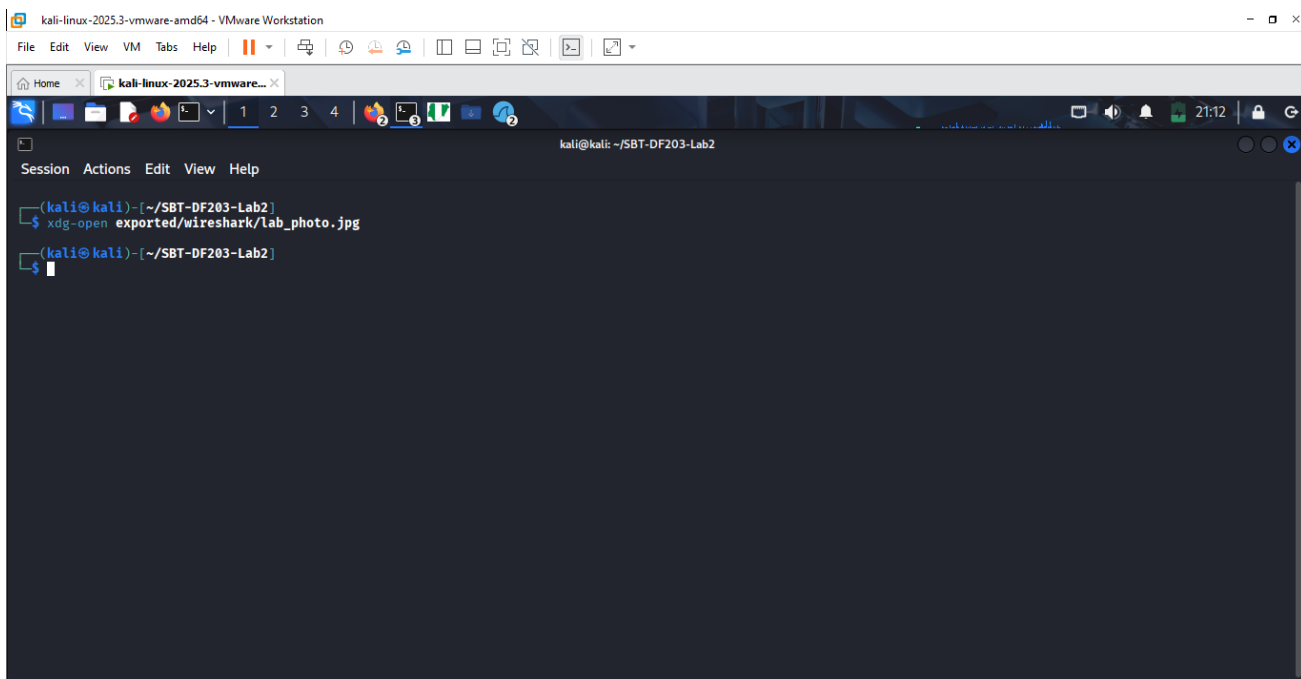


Fig 3.6 Opening recovered image.

Explanation

The screenshot shows the recovered JPEG being opened from the export directory. This was a practical usability check to confirm that the recovered file could be interpreted correctly by an image viewer.

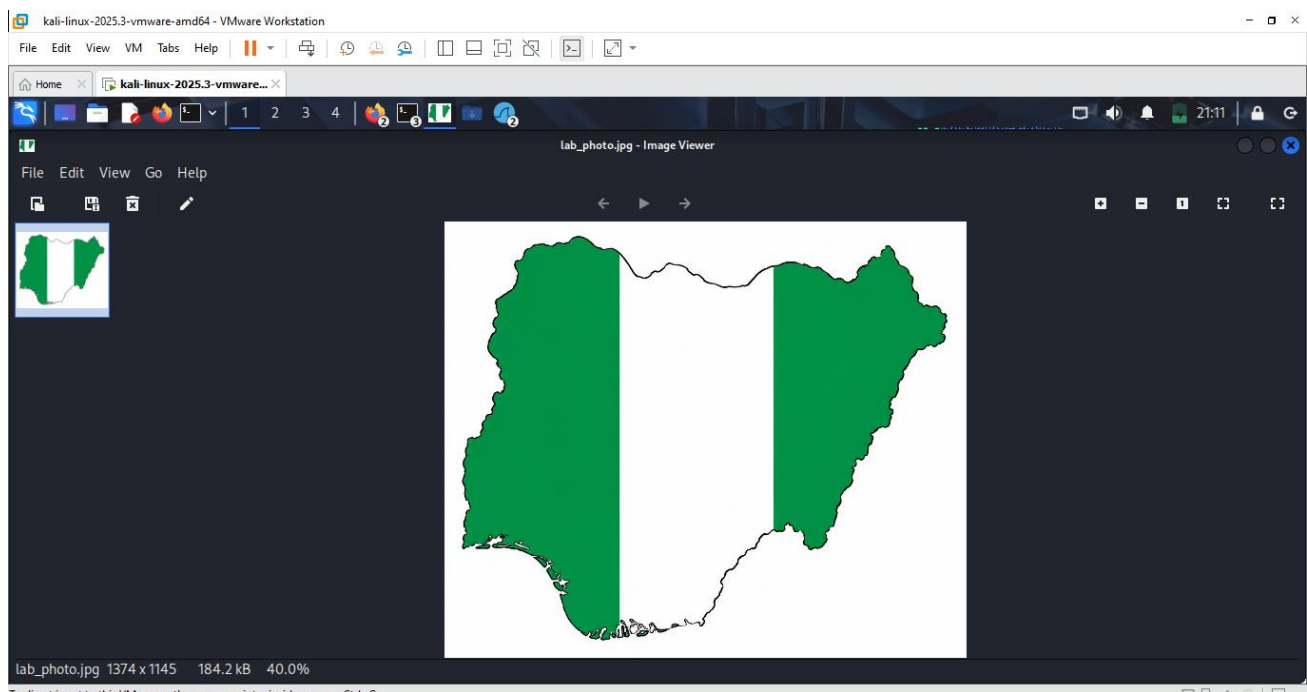


Fig 3.7 Recovered image opened successful.

Explanation

This figure visually confirms that the exported JPEG opens normally and displays the expected Nigeria image. The viewer also shows the expected 1374 × 1145 dimensions, supporting the file-type and hash evidence.

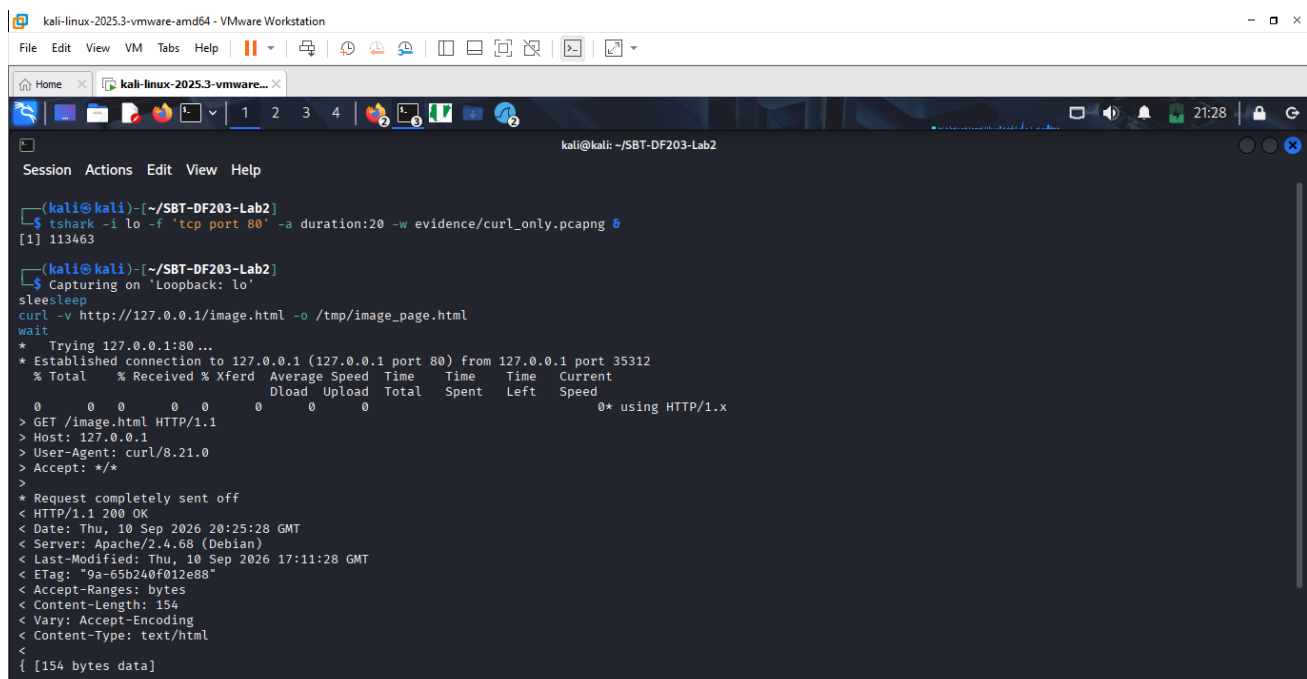


Fig 3.8 curl versus browser.

Explanation

The screenshot shows the beginning of the separate curl-only capture and the execution of the curl request for image.html. This isolated the behaviour of curl from the earlier Firefox browser activity.

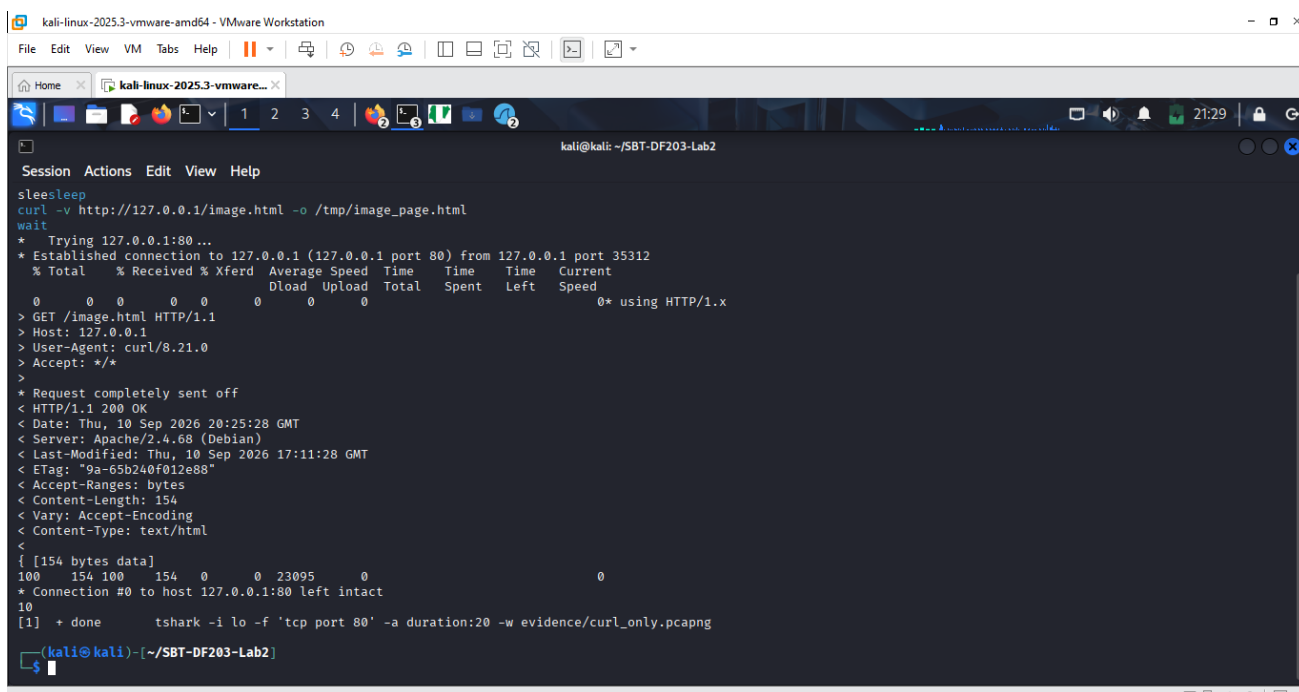


Fig 3.9 curl versus browser page 1.

Explanation

This figure shows curl sending GET /image.html and receiving an HTTP/1.1 200 OK response from Apache. The transaction demonstrates that curl successfully downloaded the HTML document without operating as a full web-browser renderer.

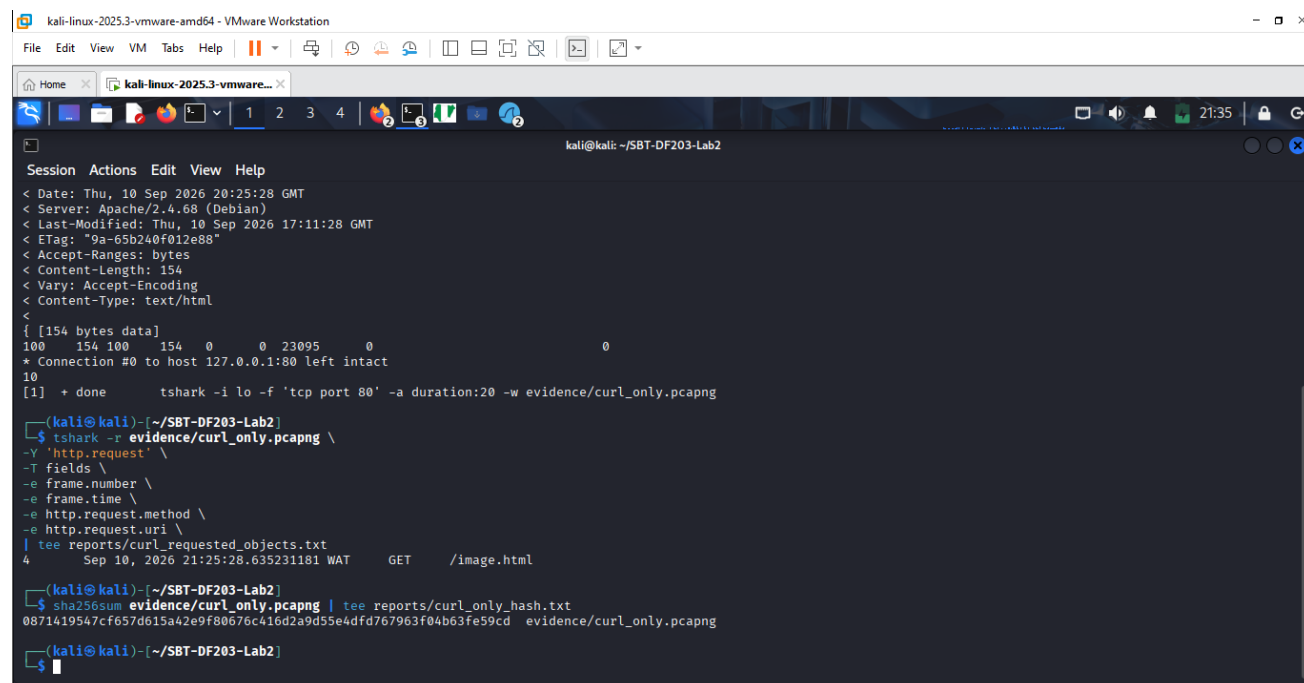


Fig 4.0 Verifying the capture.

Explanation

The final screenshot shows TShark filtering the curl-only capture and finding only GET /image.html. The

absence of /lab_photo.jpg confirms that curl did not automatically request the image referenced inside the HTML document.

Required finding	Your result
Browser HTTP requests	3 total
Required HTML request	GET /image.html
HTML request frame	4
HTML request time	20:07:00.766361217 WAT
Required image request	GET /lab_photo.jpg
Image request frame	8
Image request time	20:07:00.932501697 WAT
Additional browser request	GET /favicon.ico
Image response frame	17
HTML status	200 OK
Image status	200 OK
Favicon status	404 Not Found
Image content type	image/jpeg
Image Content-Length	184,245 bytes
Image dimensions	1374 × 1145
Reassembled TCP segments	5
TCP payload lengths	32,768; 32,768; 47,168; 47,168; 24,663 bytes
Sum of TCP payload	184,535 bytes
HTTP headers within reassembled payload	290 bytes
Original JPEG size	184,245 bytes
Extracted JPEG size	184,245 bytes
Original SHA-256	d9b0806ec7ee3f2c72e74999665780078f9d14ed 114d6bbf12dc5fe589c8b6fc
Extracted SHA-256	same hash
Integrity conclusion	Byte-for-byte identical
Browser behaviour	Requests HTML, then automatically requests embedded JPEG; also requested favicon

curl behaviour

Requested only /image.html

During the browser PCAP analysis, TShark showed:

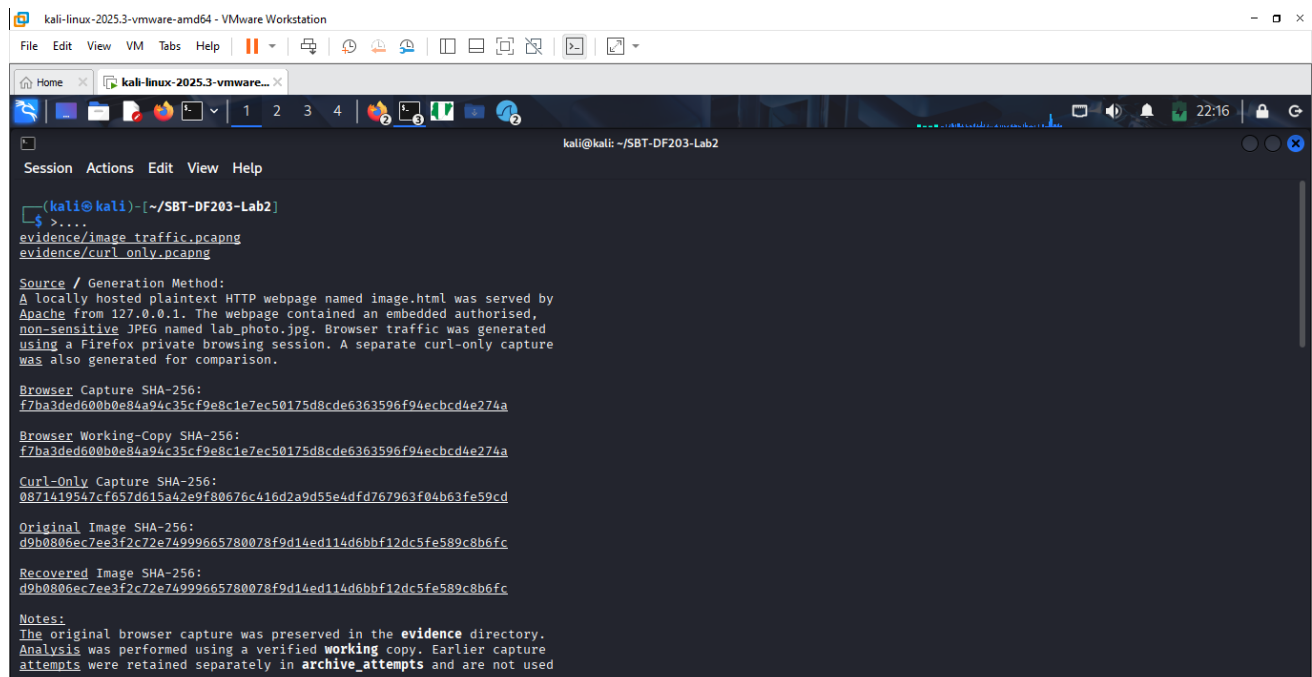
Content-Length: 155

for the HTML response, while:

the actual image.html file is 154 bytes,

current curl -I reports 154, and

Wireshark Export Objects recovered image.html as 154 bytes.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$>....
evidence/image_traffic.pcapng
evidence/curl_only.pcapng

Source / Generation Method:
A locally hosted plaintext HTTP webpage named image.html was served by
Apache from 127.0.0.1. The webpage contained an embedded authorised,
non-sensitive JPEG named lab_photo.jpg. Browser traffic was generated
using a Firefox private browsing session. A separate curl-only capture
was also generated for comparison.

Browser Capture SHA-256:
f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Browser Working-Copy SHA-256:
f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Curl-Only Capture SHA-256:
0871419547cf657d615a42e9f80676c416d2a9d55e4dfd767963f04b63fe59cd

Original Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Recovered Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Notes:
The original browser capture was preserved in the evidence directory.
Analysis was performed using a verified working copy. Earlier capture
attempts were retained separately in archive_attempts and are not used
```

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$ cat reports/chain_of_custody.txt
SBT-DF203 LAB 2 - CHAIN OF CUSTODY / MINI EVIDENCE WORKSHEET

Case/Lab Identifier:
SBT-DF203-Lab2-Athanasius-Alekwe

Trainee Name:
Athanasius Alekwe

Date:
10 September 2026

Environment:
Kali Linux 2025.3 running in VMware Workstation

Network/Capture Location:
Local loopback interface (lo), 127.0.0.1

Web Server:
Apache HTTP Server on TCP port 80

Evidence Files:
evidence/image_traffic.pcapng
evidence/curl_only.pcapng

Source / Generation Method:
A locally hosted plaintext HTTP webpage named image.html was served by
Apache from 127.0.0.1. The webpage contained an embedded authorised,
non-sensitive JPEG named lab_photo.jpg. Browser traffic was generated
using a Firefox private browsing session. A separate curl-only capture
was also generated for comparison.
```

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
evidence/curl_only.pcapng

Source / Generation Method:
A locally hosted plaintext HTTP webpage named image.html was served by
Apache from 127.0.0.1. The webpage contained an embedded authorised,
non-sensitive JPEG named lab_photo.jpg. Browser traffic was generated
using a Firefox private browsing session. A separate curl-only capture
was also generated for comparison.

Browser Capture SHA-256:
f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Browser Working-Copy SHA-256:
f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Curl-Only Capture SHA-256:
0871419547cf657d615a42e9f80676c416d2a9d5e4dfd767963f04b63fe59cd

Original Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Recovered Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Notes:
The original browser capture was preserved in the evidence directory.
Analysis was performed using a verified working copy. Earlier capture
attempts were retained separately in archive_attempts and are not used
as the final assessed evidence.

(kali@kali)~[~/SBT-DF203-Lab2]
$
```



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$>....
290 bytes
Calculation:
184535 bytes reassembled TCP payload
- 184245 bytes JPEG object
= 290 bytes HTTP response headers
14. Original Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc
15. Extracted Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc
16. Hash Comparison Conclusion:
MATCH. The original and recovered JPEG files are byte-for-byte identical.
17. Browser Behaviour:
The browser requested image.html, parsed the HTML, discovered the
embedded lab_photo.jpg resource and generated a second HTTP GET request.
Firefox also generated an additional request for favicon.ico.
18. curl Behaviour:
curl requested only /image.html. It did not automatically request
/lab_photo.jpg because curl retrieves the explicitly requested resource
but does not operate as a browser rendering engine.
19. Overall Conclusion:
The plaintext HTTP transaction was successfully captured and analysed.
The embedded JPEG was reconstructed from five TCP segments, exported
```

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(kali@kali)~[~/SBT-DF203-Lab2]
$ cat reports/forensic_findings.txt
SBT-DF203 LAB 2 - REQUIRED FORENSIC FINDINGS
Trainee: Athanasius Alekwe
1. Number of Browser HTTP Requests:
3
2. HTML Request:
GET /image.html
HTML Request Frame:
4
HTML Request Timestamp:
Sep 10, 2026 20:07:00.766361217 WAT
3. Image Request:
GET /lab_photo.jpg
Image Request Frame:
8
Image Request Timestamp:
Sep 10, 2026 20:07:00.932501697 WAT
4. Additional Browser Request:
GET /favicon.ico
Favicon Request Frame:
19
```

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
5. HTTP Response Codes:
image.html - 200 OK
lab_photo.jpg - 200 OK
favicon.ico - 404 Not Found
6. Image Response Frame:
17
7. Image Content Type:
image/jpeg
8. Image Content-Length:
184245 bytes
9. Original Image Dimensions:
1374 x 1145 pixels
10. Number of Reassembled TCP Segments:
5
11. TCP Payload Lengths:
Frame 9 : 32768 bytes
Frame 11 : 32768 bytes
Frame 13 : 47168 bytes
Frame 14 : 47168 bytes
Frame 17 : 24663 bytes
12. Sum of TCP Payload Lengths:
184535 bytes
13. HTTP Response Header Size:
290 bytes
```

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
290 bytes
Calculation:
184535 bytes reassembled TCP payload
- 184245 bytes JPEG object
= 290 bytes HTTP response headers
14. Original Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc
15. Extracted Image SHA-256:
d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc
16. Hash Comparison Conclusion:
MATCH. The original and recovered JPEG files are byte-for-byte identical.
17. Browser Behaviour:
The browser requested image.html, parsed the HTML, discovered the
embedded lab_photo.jpg resource and generated a second HTTP GET request.
Firefox also generated an additional request for favicon.ico.
18. curl Behaviour:
curl requested only /image.html. It did not automatically request
/lab_photo.jpg because curl retrieves the explicitly requested resource
but does not operate as a browser rendering engine.
19. Overall Conclusion:
The plaintext HTTP transaction was successfully captured and analysed.
The embedded JPEG was reconstructed from five TCP segments, exported
from the capture and verified using SHA-256 as identical to the source.
(kali@kali)~[~/SBT-DF203-Lab2]
```

LIMITATIONS

The main limitation was that the experiment used the local loopback interface rather than a physical network connection. Because of this, TCP segmentation observed in TShark may not represent the exact segment sizes that would appear on an Ethernet link. Operating-system buffering and offloading can significantly affect the way localhost traffic appears at the capture point.

An additional /favicon.ico request was generated automatically by Firefox. This was retained as part of the evidence and documented rather than removed. The resulting 404 Not Found response did not interfere with the successful HTML or JPEG transfers.

A minor HTML Content-Length discrepancy was also observed. During analysis of the browser PCAP, TShark displayed Content-Length: 155 for the HTML response, whereas the actual image.html file, the curl verification and Wireshark's exported HTML object showed 154 bytes. This difference was documented rather than altered, and it does not affect the JPEG integrity result.

MINI EVIDENCE AND CHAIN-OF-CUSTODY WORKSHEET

The official worksheet requires the case identifier, trainee, starting time, evidence files, generation method, original and working-copy hashes, workstation, and notes concerning changes.

Evidence Field	Entry
Case/Lab Identifier	SBT-DF203-Lab2-Athanasius-Alekwe
Trainee Name	Athanasius Orikeze Alekwe
Registration Number	2025/FWSD/11230

Date and Time Started	2026-09-10T17:40:44+01:00
Main Evidence File	evidence/image_traffic.pcapng
Additional Evidence File	evidence/curl_only.pcapng
Working Copy	working/image_traffic_working.pcapng
Source Objects	/var/www/html/image.html and /var/www/html/lab_photo.jpg
Recovered Objects	exported/http_objects/lab_photo.jpg and exported/wireshark/lab_photo.jpg
Source / Generation Method	Plaintext HTTP page hosted locally using Apache on 127.0.0.1:80. Firefox private browsing generated the main browser traffic; curl generated the separate comparison capture.

Original Browser PCAP SHA-256

f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Working-Copy SHA-256

f7ba3ded600b0e84a94c35cf9e8c1e7ec50175d8cde6363596f94ecbcd4e274a

Curl Capture SHA-256

0871419547cf657d615a42e9f80676c416d2a9d55e4dfd767963f04b63fe59cd

Original Image SHA-256

d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Recovered Image SHA-256

d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc

Analysis Workstation / VM	Kali Linux 2025.3 running in VMware Workstation
---------------------------	---

Capture Interface	Loopback interface (lo)
-------------------	-------------------------

Notes on Changes

The original browser PCAP was preserved and analysis was performed on a verified working copy. Earlier attempts were retained separately under archive_attempts and were not used as the final assessed evidence.

REQUIRED FORENSIC FINDINGS

Required Finding	Final Observation
------------------	-------------------

Number of browser HTTP requests	3
------------------------------------	---

HTML request URI	GET /image.html
------------------	-----------------

HTML request frame	4
--------------------	---

Required Finding	Final Observation
HTML request timestamp	20:07:00.766361217 WAT
Image request URI	GET /lab_photo.jpg
Image request frame	8
Image request timestamp	20:07:00.932501697 WAT
Additional browser request	GET /favicon.ico, frame 19
HTML response	200 OK
Image response	200 OK
Favicon response	404 Not Found
Image response frame	17
Image Content-Type	image/jpeg
Image Content-Length	184,245 bytes
Image dimensions	1374 × 1145 pixels
Number of reassembled TCP segments	5
TCP payload lengths	32,768; 32,768; 47,168; 47,168; 24,663 bytes
Sum of TCP payload lengths	184,535 bytes
HTTP headers inside reassembled payload	290 bytes
Original JPEG size	184,245 bytes
Recovered JPEG size	184,245 bytes
Original SHA-256	d9b0806ec7ee3f2c72e74999665780078f9d14ed114d6bbf12dc5fe589c8b6fc
Recovered SHA-256	Same as original

Required Finding	Final Observation
Hash comparison	Match — byte-for-byte identical
Browser behaviour	Requested the HTML, then automatically requested the referenced JPEG; Firefox also requested a favicon.
curl behaviour	Requested only /image.html; no automatic JPEG request was generated.

Conclusion

This practical gave me a clear understanding of what actually happens behind the scenes when a browser loads a simple webpage containing an image. Although the page appeared as one item in Firefox, the packet capture showed that the browser generated separate HTTP requests for image.html and lab_photo.jpg, as well as an additional request for favicon.ico. This demonstrated that a browser first retrieves the HTML document, reads its contents, and then requests other resources referenced inside the page.

The image transfer also showed how a larger HTTP object can be divided across several TCP segments before being reassembled at the receiving end. In my capture, the JPEG response was reconstructed from five TCP segments, producing a total TCP payload of 184,535 bytes. The image itself was 184,245 bytes, with the remaining 290 bytes representing the HTTP response header information.

Most importantly, I was able to successfully recover the transferred JPEG from the packet capture using both TShark and Wireshark. The SHA-256 hash of the recovered image matched the original image exactly, confirming that the recovered file was byte-for-byte identical and had not been altered or corrupted during the transfer or extraction process.

The curl comparison further helped show the difference between a browser and a basic HTTP client. While Firefox automatically requested the embedded image after reading the HTML, curl requested only image.html because it does not normally process and render webpage resources in the same way as a browser. Overall, this lab strengthened my practical understanding of HTTP traffic, browser behaviour, TCP segmentation and reassembly, packet analysis, object recovery, and the importance of hash verification when handling digital forensic evidence.

Screenshots

Fig 1.0 Lab folder structure and start time.

Fig 1.1 Verifying imagemagick and wireshark are installed.

Fig 1.2 Verifying tshark is installed.

Fig 1.3 Starting Apache page.

Fig 1.4 Choosing the image and copying to Apache's web folder.

Fig 1.5 Creating the HTML page and verifying both files.

Fig 1.6 Changing the permission of the image file.

Fig 1.7 Webpage embedded image.

Fig 1.8 Webpage embedded image page 1.

Fig 1.9 image file details.

Fig 2.0 Generating the baseline SHA-256 hashes.

Fig 2.1 Verifying both HTTP objects.

Fig 2.2 Capturing on 'Loopback'.

Fig 2.3 Capturing the traffic.

Fig 2.4 24 packets captured.

Fig 2.5 Browser capture and pcap hashes.

Fig 2.6 Recording the HTTP request/response.

Fig 2.7 Capturing HTTP headers.

Fig 2.8 Capturing tshark details.

Fig 2.9. Capturing tshark details page 1.

Fig 3.0 Exporting the transferred JPEG.

Fig 3.0 Exporting the transferred JPEG.

Fig 3.1 SHA-256 integrity test.

Fig 3.2 Opening Wireshark to perform manual.

Fig 3.3 Opening Wireshark—Export--HTTP.

Fig 3.4 Saving to /SBT-DF203-Lab2/exported/wireshark/lab_photo.jpg

Fig 3.5 Verifying the GUI-exported version.

Fig 3.6 Opening recovered image.

Fig 3.7 Recovered image opened successful.

Fig 3.8 curl versus browser.

Fig 3.9 curl versus browser page 1.

Fig 4.0 Verifying the capture.

References

ICDFA Materials.